<table>
<tr><td colspan="2">NAME:

MUHAMMAD 'ARIF NAQYYUDDIN BIN ABD KAHAR
HAIROLAKMA DANISH BIN HAIRIOLNIZAM
AHMAD HAFIQ DANIEL BIN RODUAN
PUTERA AIDIL BIN SHARBUDIN</td><td rowspan="2">STUDENT ID:

(AM2508020305)
(AM2505018962)
(AM2508019635)
(AM2508020021)</td></tr>
<tr><td colspan="2">SECTION: 05</td></tr>
<tr><td colspan="2">LECTURER:

Madam NURAIN NABILA BINTI BAHAROM</td><td>LAB GROUP / TUTORIAL GROUP / TUTOR (IF APPLICABLE):</td></tr>
<tr><td colspan="2">COURSE & COURSE CODE:

WEB API DEVELOPMENT (SWC3633)</td><td>SUBMISSION DATE:

14/10/2026</td></tr>
<tr><td colspan="2">ASSIGNMENT NO. / TITLE:

Development of a RESTful Web API for an Event Ticketing and Venue Booking System</td><td>EXTENSION & LATE SUBMISSION:

Disallowed</td></tr>
<tr><td>ASSIGNMENT TYPE:

GROUPING</td><td>% OF ASSIGNMENT:

50%</td><td>RETURNING DATE:</td></tr>
<tr><td colspan="3">PENALTIES:

1. 10% of the original mark will be deducted for every one-week period after the submission date. No work will be accepted after two weeks of the deadline.

2. If you were unable to submit the coursework on time due to extenuating circumstances you may be eligible for an extension.

3. Extension will not exceed one week.</td></tr>
<tr><td colspan="3">Declaration: I/we the undersigned confirm that I/we have read and agree to abide by these regulations on plagiarism and cheating. I/we confirm that this piece of work is my/our own. I/we consent to appropriate storage of our work for checking to ensure that there is no plagiarism/ academic cheating.

Signature(s):</td></tr>
</table>

PROJECT

BOOM+ Sports Event Ticketing & Venue Booking

A group report submitted by

MUHAMMAD 'ARIF NAQYYUDDIN BIN ABD KAHAR

(AM2505018962)

HAIROLAKMA DANISH BIN HAIRIOLNIZAM
(AM2508020305)

AHMAD HAFIQ DANIEL BIN RODUAN
(AM2508019635)

PUTERA AIDIL BIN SHARBUDIN

(AM2508020021)

Lecture:

MDM NURAIN NABILA

An assessment submitted in partial fulfillment of the requirements for the course

SWC 3633 WEB API DEVELOPMENT

UNIVERSITI POLY-TECH MALAYSIA

Table of Content

1. SYSTEM OVERVIEW..... 10
    1.1 SYSTEM DESCRIPTION..... 10
    1.2 OBJECTIVES..... 10
    1.3 INTENDED USERS..... 11
    1.4 MAIN FEATURES..... 11
    1.5 BOOKING WORKFLOW (BPMN)..... 12
2. DATABASE DESIGN..... 13
    2.1 ENTITY RELATIONSHIP DIAGRAM (ERD)..... 13
    2.2 DATA DICTIONARY..... 14
    2.3 DATA DEFINITION LANGUAGE (DDL)..... 15
    2.4 DATA MANIPULATION LANGUAGE (DML)..... 19
3. RESTful API DOCUMENTATION..... 23
    3.1 Authentication..... 23
    3.2 Users..... 26
    3.3 Venues and Courts..... 29
    3.4 Events and Ticket Types..... 33
    3.5 Bookings and Payments..... 37
    3.6 System, Statistics and Logs..... 40
4. API TESTING..... 43
    4.1 TESTING SETUP..... 43
    4.2 EXPORTED POSTMAN COLLECTION..... 43
    4.3 SUCCESSFUL RESPONSES..... 44
    4.4 ERROR RESPONSES..... 48
5. SYSTEM IMPLEMENTATION..... 53
    5.1 SHARED BACKEND IMPLEMENTATION..... 53
    5.2 HOW THE FRONTENDS COMMUNICATE WITH THE API..... 60
    5.3 FRONTEND OF HAFIQ: Authentication, User Profile and User Administration..... 62
    5.4 FRONTEND OF AKMA: Organiser / Admin Management Dashboard..... 65
    5.5 FRONTEND OF ARIF: Customer Booking & Checkout Portal..... 71
    5.6 FRONTEND OF PUTERA AIDIL: Customer Account & Ticketing Portal..... 74
6. SYSTEM DEMONSTRATION..... 76
        6.1 Login and role-based dashboards..... 76
        6.2 Browsing events and live availability..... 77
        6.3 Booking a ticket and QR e-ticket..... 78
        6.4 Booking a court..... 79
        6.6 Admin: user management and logs..... 79
        6.7 Cancellation and refund..... 80
7. PROJECT REPOSITORY..... 81
8. ADDITIONAL / ADVANCED FEATURES..... 82

8.1 SECURITY..... 82
8.2 MIDDLEWARE AND API MANAGEMENT..... 83
8.3 THIRD-PARTY API: QR CODE GENERATOR..... 84
8.4 BUSINESS LOGIC FEATURES..... 85
9. DEBUGGING AND PERFORMANCE OPTIMISATION..... 86
9.1 IMPLEMENTATION ISSUES, ROOT CAUSES AND SOLUTIONS..... 86
9.2 PERFORMANCE TESTING..... 87
10. INDIVIDUAL REFLECTION AND CONTRIBUTION RECORD..... 88
10.1 CONTRIBUTION RECORD..... 88
10.2 INDIVIDUAL REFLECTIONS..... 89
10.2 HAFIQ..... 89
10.2 AKMA..... 90
10.2 ARIF..... 91
10.2 PUTERA AIDIL..... 92

LIST OF FIGURES

Figure 1.1: BPMN of the BOOM+ booking process — Page 12
Figure 2.1: Entity Relationship Diagram of boom_plus — Page 13
Figure 4.1: POST /auth/login (200 OK) — Page 46
Figure 4.2: GET /venues with filters (200 OK) — Page 46
Figure 4.3: POST /events (201 Created) — Page 47
Figure 4.4: GET /events with sort and limit (200 OK) — Page 47
Figure 4.5: POST /bookings (201 Created with QR code) — Page 48
Figure 4.6: POST /bookings/{id}/cancel (200 OK) — Page 48
Figure 4.7: GET /system/health (200 OK) — Page 48
Figure 4.8: POST /auth/login wrong password (401) — Page 49
Figure 4.9: GET /users as customer (403) — Page 50
Figure 4.10: GET /bookings without token (401) — Page 50
Figure 4.11: POST /bookings validation error (422) — Page 51
Figure 4.12: POST /bookings sold out (409) — Page 51
Figure 4.13: POST /bookings court slot taken (409) — Page 52
Figure 4.14: POST /events overlapping event (409) — Page 52
Figure 4.15: GET /events/99999 (404) — Page 53
Figure 4.16: 16th login request (429 Too Many Requests) — Page 53
Figure 5.2: Hafiq: login page — Page 64
Figure 5.3: Hafiq: user management page (admin) — Page 64
Figure 5.4: Hafiq: audit log page (admin) — Page 65
Figure 5.5: Organiser Workspace Dashboard — Page 67
Figure 5.6: Event Scheduling & Management — Page 68
Figure 5.7: Venue Management — Page 69
Figure 5.8: Reservations & Attendance Tracking — Page 70
Figure 5.9: User Profile & Role Verification — Page 71
Figure 5.10: Arif: event booking page — Page 73
Figure 5.11: Arif: e-ticket with QR code — Page 73
Figure 5.12: Arif: organiser check-in page — Page 74
Figure 5.13: Putera Aidil: booking history page — Page 75
Figure 5.14: Putera Aidil: modify booking page — Page 76
Figure 5.15: Putera Aidil: API request log (admin) — Page 76
Figure 6.1: Login and role-based dashboards — Page 77
Figure 6.2: Browsing events and live availability — Page 78

Figure 6.3: Booking a ticket and QR e-ticket — Page 79

Figure 6.4: Booking a court — Page 80

Figure 6.6: Admin: user management and logs — Page 80

Figure 6.7: Cancellation and refund — Page 81

LIST OF TABLES

Main Report Tables

Table 1.1: Intended Users and System Roles — Page 11

Table 2.1: Entity Relationships and Cardinality Mapping — Page 14

Table 2.2: Data Dictionary for boom_plus Database Schema — Page 15

Table 3.1: Standard API HTTP Status Codes and Descriptions — Page 24

Table 4.1: Exported Postman Test Collection Endpoints and Methods — Page 45

Table 4.2: Postman Test Suite - Successful Response Test Cases — Page 45

Table 4.3: Postman Test Suite - Error and Validation Test Cases — Page 49

Table 5.1: Backend Directory Structure and Component Responsibilities — Page 54

Table 5.2: Frontend-to-API Communication Lifecycle Steps — Page 63

Table 5.3: Hafiq Frontend Portal Files and Mapped API Endpoints — Page 63

Table 5.4: Akma Organiser Dashboard Files and Mapped API Endpoints — Page 66

Table 5.5: Arif Customer Booking Portal Files and Mapped API Endpoints — Page 72

Table 5.6: Putera Aidil Account and Logging Portal Files and Mapped API Endpoints — Page 75

Table 7.1: Course Source Code and Submission Deliverables Checklist — Page 82

Table 8.1: Middleware Implementations and API Management Services — Page 84

Table 9.1: Technical Implementation Issues, Root Causes, and Applied Solutions — Page 87

Table 9.2: API Response Time and EXPLAIN Query Performance Optimization Results — Page 88

Table 10.1: Group Member Responsibilities and Individual Contribution Record — Page 89

API Endpoint Specifications (Section 3)

Table 3.2: API Endpoint Specification - POST /auth/register — Page 25

Table 3.3: API Endpoint Specification - POST /auth/login — Page 25

Table 3.4: API Endpoint Specification - GET /auth/me — Page 26

Table 3.5: API Endpoint Specification - POST /auth/forgot-password — Page 26

Table 3.6: API Endpoint Specification - POST /auth/reset-password — Page 26
Table 3.7: API Endpoint Specification - GET /users/me — Page 27
Table 3.8: API Endpoint Specification - PUT /users/me — Page 27
Table 3.9: API Endpoint Specification - PUT /users/me/password — Page 27
Table 3.10: API Endpoint Specification - GET /users — Page 28
Table 3.11: API Endpoint Specification - POST /users — Page 28
Table 3.12: API Endpoint Specification - GET /users/{id} — Page 29
Table 3.13: API Endpoint Specification - PUT /users/{id}/role — Page 29
Table 3.14: API Endpoint Specification - PUT /users/{id}/status — Page 29
Table 3.15: API Endpoint Specification - DELETE /users/{id} — Page 30
Table 3.16: API Endpoint Specification - GET /venues — Page 30
Table 3.17: API Endpoint Specification - POST /venues — Page 30
Table 3.18: API Endpoint Specification - GET /venues/{id} — Page 31
Table 3.19: API Endpoint Specification - PUT/PATCH /venues/{id} — Page 31
Table 3.20: API Endpoint Specification - DELETE /venues/{id} — Page 32
Table 3.21: API Endpoint Specification - GET /venues/{id}/courts — Page 32
Table 3.22: API Endpoint Specification - POST /venues/{id}/courts — Page 33
Table 3.23: API Endpoint Specification - PUT/PATCH /venues/{id}/courts/{courtId} — Page 33
Table 3.24: API Endpoint Specification - DELETE /venues/{id}/courts/{courtId} — Page 34
Table 3.25: API Endpoint Specification - GET /venues/{id}/schedule — Page 34
Table 3.26: API Endpoint Specification - GET /events — Page 34
Table 3.27: API Endpoint Specification - POST /events — Page 35
Table 3.28: API Endpoint Specification - GET /events/{id} — Page 35
Table 3.29: API Endpoint Specification - PUT/PATCH /events/{id} — Page 36
Table 3.30: API Endpoint Specification - DELETE /events/{id} — Page 36
Table 3.31: API Endpoint Specification - GET /events/{id}/tickets — Page 36
Table 3.32: API Endpoint Specification - POST /events/{id}/tickets — Page 37
Table 3.33: API Endpoint Specification - PUT/PATCH /events/{id}/tickets/{ticketId} — Page 37
Table 3.34: API Endpoint Specification - DELETE /events/{id}/tickets/{ticketId} — Page 37
Table 3.35: API Endpoint Specification - GET /events/{id}/availability — Page 38
Table 3.36: API Endpoint Specification - GET /bookings — Page 38
Table 3.37: API Endpoint Specification - POST /bookings — Page 39
Table 3.38: API Endpoint Specification - GET /bookings/{id} — Page 39
Table 3.39: API Endpoint Specification - PUT/PATCH /bookings/{id} — Page 40
Table 3.40: API Endpoint Specification - POST /bookings/{id}/cancel — Page 41

Table 3.41: API Endpoint Specification - PUT/PATCH /bookings/{id}/status — Page 41
Table 3.42: API Endpoint Specification - GET /system/health — Page 42
Table 3.43: API Endpoint Specification - GET /system/stats — Page 42
Table 3.44: API Endpoint Specification - GET /system/audit-logs — Page 42
Table 3.45: API Endpoint Specification - GET /system/request-logs — Page 43

1. SYSTEM OVERVIEW

1.1 SYSTEM DESCRIPTION

BOOM+ (which stands for Book Online Optimised Management) is a web-based sport event ticketing and venue booking system for **Pickleball, Futsal and Badminton** in Malaysia. Users will be able to browse through the events, access real-time ticket availability, purchase tickets or even book sports courts. The respective organisations will be managing their courts and tickets.

Each of the four BOOM+ frontends are created by each student (Hafiq, Akma, Arif, Putera-Aidil) and they are individual PHP portals which share ONE SAME **XAMPP RESTful Web API**. Users authenticate via a JSON Web Token.

1.2 OBJECTIVES

- To build one shared RESTful API that can support all features for event booking and court booking, for the three sports in scope
- To secure the system with JWT authentication, RBAC, validation and rate limiting.
- To handle payment process and database row locking to prevent conflicting booking requests.
- To build the backend with four independent, customised front end, so that users can have customized UI/UX and role based dashboards:
  - Admin Dashboard
  - Organiser Dashboard
  - User portal
- To document the endpoints of the API and test with Postman and apply various web development concepts in practice.

1.3 INTENDED USERS

Role	Who	Core functions
Administrator	System staff	● Manage users and roles ● manage every venue, event and booking ● view audit and request logs
Organiser / Staff	Venue owners and event managers	● Manage own venues, courts, ● schedules, events and ticket capacity ● view and manage bookings ● check guests in
Customer	Players and fans	● Register and log in browse events ● check live availability ● book, modify and cancel tickets and courts ● view own tickets and history

Table 1.1: Intended Users and System Roles

1.4 MAIN FEATURES

- **Authentication and Accounts:** User Registration, Log-in (with JWT token expiration of 1 hour), Forgot and Reset Password and Profile/Password Change functionalities.
- **Venues and Courts:** CRUD operations for venues and courts including filtering by city, sport and text search and retrieval of daily venue schedules.
- **Events and Ticket Types:** CRUD functionality, event lifecycle management (draft, published, cancelled, completed), enforcement of overlap and capacity rules and real-time availability display.
- **Bookings:** Event ticket and court bookings including various payment methods like card, FPX, e-wallet and cash. It also supports modification, cancellation with automatic refunds and maintenance of a comprehensive booking status workflow.
- **E-Ticket:** Generation of a unique QR code for each booking via api. Qrserver. Com and organiser check-in using booking reference.
- **Security and Monitoring:** Implementation of RBAC, rate limiting, security headers, CSRF tokens, audit log to track server modifications and an API request log for monitoring interaction with it.
- **List Features:** Sorting and pagination on all list endpoints and includes advance filtering option.

1.5 BOOKING WORKFLOW (BPMN)

Customer logs onto system, navigates to event/court of choice, views availability, places booking & makes payment. The API authenticates and blocks stock internally in a transaction process, records payment and generates the QR Code before returning the booking. If cash is selected then payment displays pending until authenticated by an event organiser or admin. If a booking is cancelled then stock is unblocked and (if applicable) payment refunded. Event organisers will have a way to check guests in upon entry and admin can have oversight via logs.

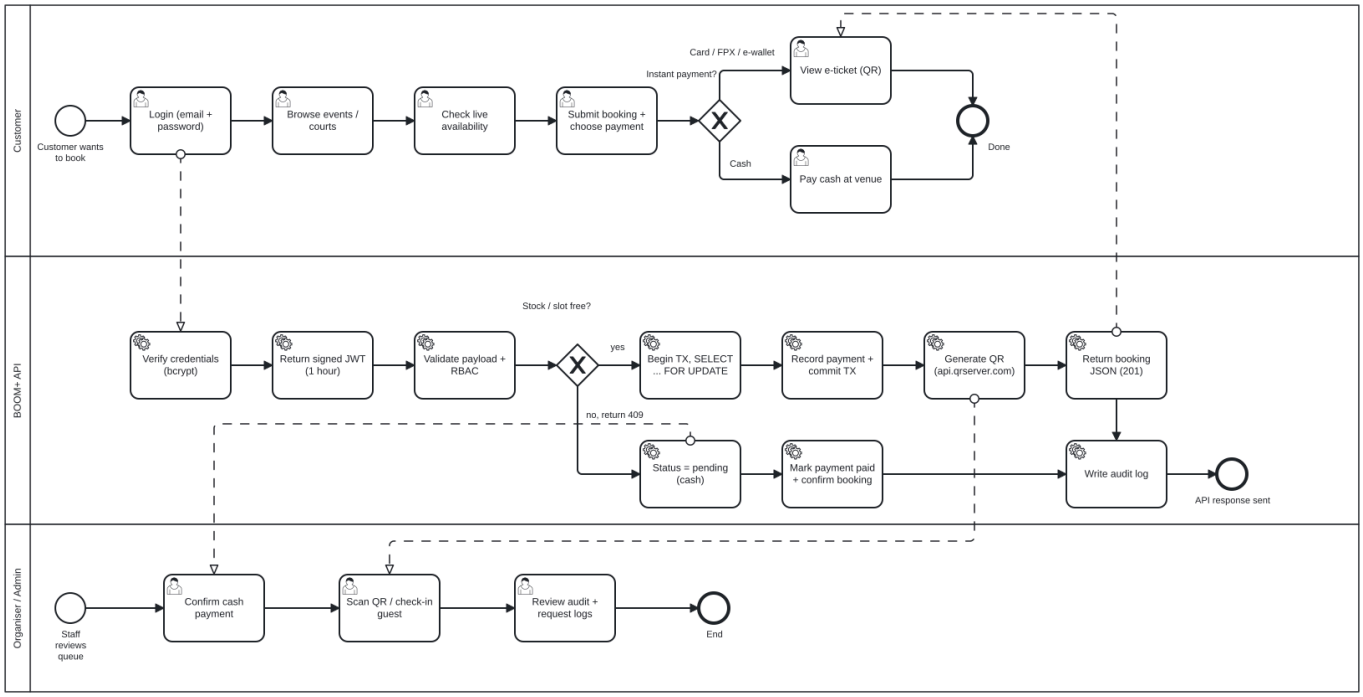


Figure 1.1: BPMN of the BOOM+ booking process

2. DATABASE DESIGN

The database boom_plus (MySQL / MariaDB, InnoDB, utf8mb4) has 11 tables. Role and sport are ENUM columns, so no three-row lookup tables are needed. Integrity is enforced with primary keys, foreign keys, UNIQUE and CHECK constraints, and every table contains 5 to 8 sample rows.

2.1 ENTITY RELATIONSHIP DIAGRAM (ERD)

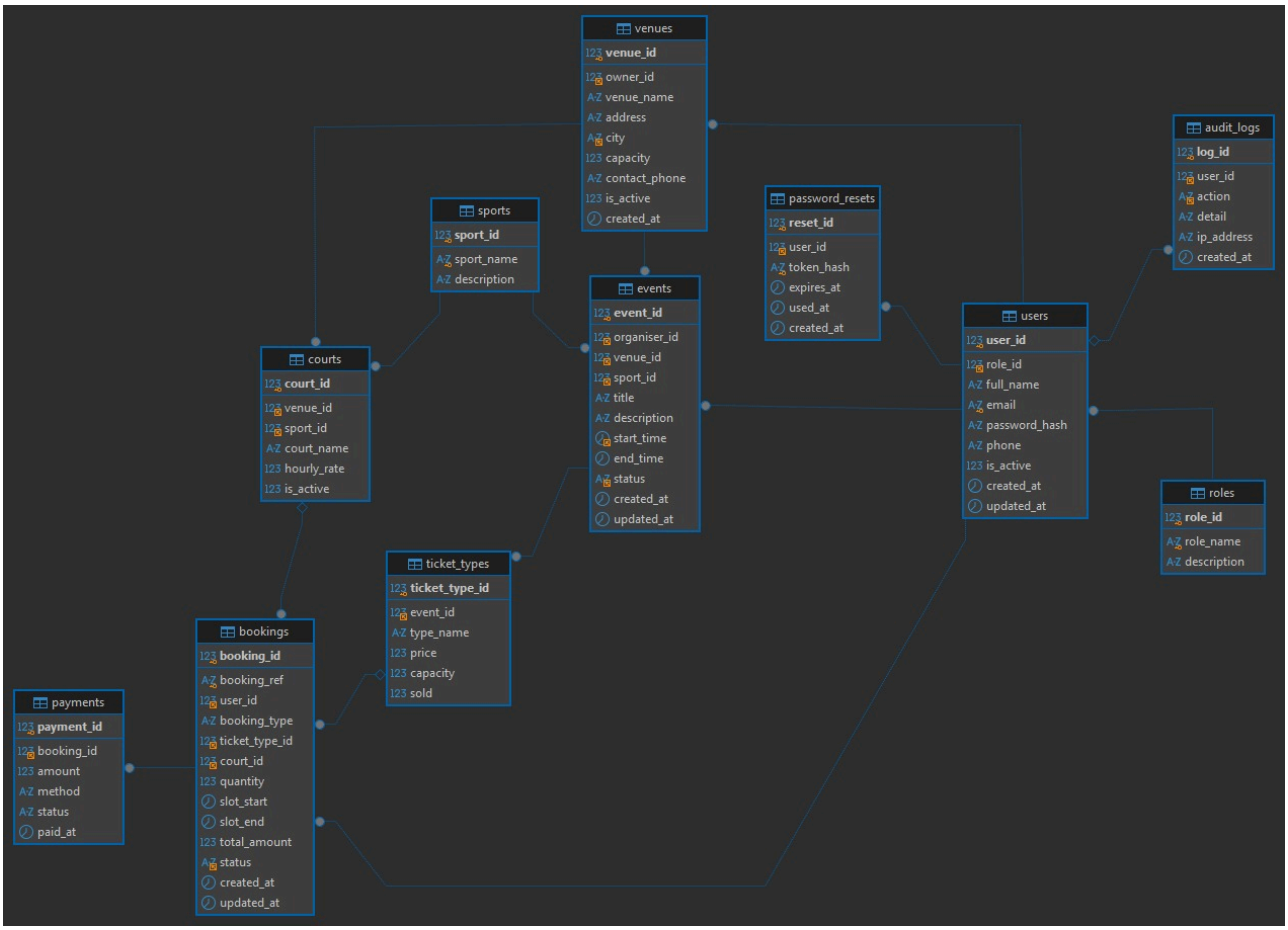


Figure 2.1: Entity Relationship Diagram of boom_plus

Relationship	Cardinality
users → password_resets	one to many (a user can request many resets)
users → venues (owner)	one to many
users → events (organiser)	one to many
users → bookings	one to many
users → audit_logs	one to many
venues → courts	one to many (cascade delete)

Relationship	Cardinality
venues → events	one to many
events → ticket_types	one to many (cascade delete)
ticket_types → bookings	one to many (event ticket bookings)
courts → bookings	one to many (court bookings)
bookings → payments	one to one (cascade delete)

Table 2.1: Entity Relationships and Cardinality Mapping

Request_logs and rate_limits are infrastructure tables written by the logging and rate-limit middleware and have no foreign keys.

2.2 DATA DICTIONARY

Table	Columns (PK = primary key, FK = foreign key)
users	`user_id PK role ENUM(admin, organiser, customer) full_name email UNIQUE password_hash phone is_active created_at updated_at`
password_resets	`reset_id PK user_id FK token_hash UNIQUE (SHA-256) expires_at used_at created_at`
venues	`venue_id PK owner_id FK venue_name address city capacity (> 0) contact_phone is_active created_at`
courts	`court_id PK venue_id FK sport ENUM court_name (unique per venue) hourly_rate (>= 0) is_active`
events	`event_id PK organiser_id FK venue_id FK sport ENUM title description start_time end_time (> start_time) status ENUM(draft, published, cancelled, completed) timestamps`
ticket_types	`ticket_type_id PK event_id FK type_name (unique per event) price (>= 0) capacity sold (CHECK sold <= capacity)`
bookings	`booking_id PK booking_ref UNIQUE user_id FK booking_type ENUM(event_ticket, court) ticket_type_id FK null court_id FK null quantity (> 0) slot_start slot_end total_amount status ENUM(pending, confirmed, cancelled, completed) qr_code timestamps`
payments	`payment_id PK booking_id FK UNIQUE amount method ENUM(card, fpx, ewallet, cash) status ENUM(pending, paid, failed, refunded) paid_at`
audit_logs	`log_id PK user_id FK null action detail ip_address created_at`
request_logs	`request_id PK user_id method path status_code duration_ms ip_address created_at`

Table	Columns (PK = primary key, FK = foreign key)
rate_limits	`bucket + window_start (composite PK) hits`

Table 2.2: Data Dictionary for boom_plus Database Schema

2.3 DATA DEFINITION LANGUAGE (DDL)

The DDL below is from database/boom_plus.sql and creates the database, tables, keys, indexes and CHECK constraints.

```
SQL
CREATE DATABASE boom_plus CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci
USE boom_plus

CREATE TABLE users (
  user_id       INT UNSIGNED NOT NULL AUTO_INCREMENT,
  role          ENUM('admin','organiser','customer') NOT NULL DEFAULT 'customer',
  full_name     VARCHAR(100) NOT NULL,
  email         VARCHAR(150) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  phone         VARCHAR(20)  NULL,
  is_active     TINYINT(1)   NOT NULL DEFAULT 1,
  created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  PRIMARY KEY (user_id),
  UNIQUE KEY uq_users_email (email),
  KEY idx_users_role (role)
) ENGINE=InnoDB

CREATE TABLE password_resets (
  reset_id   INT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id    INT UNSIGNED NOT NULL,
  token_hash CHAR(64)     NOT NULL,
  expires_at DATETIME     NOT NULL,
  used_at    DATETIME     NULL,
  created_at TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (reset_id),
  UNIQUE KEY uq_reset_token (token_hash),
  KEY idx_reset_user (user_id),
  CONSTRAINT fk_reset_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON
DELETE CASCADE
) ENGINE=InnoDB

CREATE TABLE venues (
  venue_id      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  owner_id      INT UNSIGNED NOT NULL,
  venue_name    VARCHAR(120) NOT NULL,
  address       VARCHAR(255) NOT NULL,
  city          VARCHAR(80)  NOT NULL,
  capacity      INT UNSIGNED NOT NULL,
```

```
  contact_phone VARCHAR(20)  NULL,
  is_active     TINYINT(1)   NOT NULL DEFAULT 1,
  created_at    TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (venue_id),
  KEY idx_venues_owner (owner_id),
  KEY idx_venues_city (city),
  CONSTRAINT fk_venues_owner FOREIGN KEY (owner_id) REFERENCES users (user_id),
  CONSTRAINT chk_venue_capacity CHECK (capacity > 0)
) ENGINE=InnoDB

CREATE TABLE courts (
  court_id    INT UNSIGNED NOT NULL AUTO_INCREMENT,
  venue_id    INT UNSIGNED NOT NULL,
  sport       ENUM('Pickleball','Futsal','Badminton') NOT NULL,
  court_name  VARCHAR(60)  NOT NULL,
  hourly_rate DECIMAL(8,2) NOT NULL,
  is_active   TINYINT(1)   NOT NULL DEFAULT 1,
  PRIMARY KEY (court_id),
  UNIQUE KEY uq_court_per_venue (venue_id, court_name),
  KEY idx_courts_sport (sport),
  CONSTRAINT fk_courts_venue FOREIGN KEY (venue_id) REFERENCES venues (venue_id)
ON DELETE CASCADE,
  CONSTRAINT chk_court_rate CHECK (hourly_rate >= 0)
) ENGINE=InnoDB

CREATE TABLE events (
  event_id     INT UNSIGNED NOT NULL AUTO_INCREMENT,
  organiser_id INT UNSIGNED NOT NULL,
  venue_id     INT UNSIGNED NOT NULL,
  sport        ENUM('Pickleball','Futsal','Badminton') NOT NULL,
  title        VARCHAR(150) NOT NULL,
  description  TEXT NULL,
  start_time   DATETIME NOT NULL,
  end_time     DATETIME NOT NULL,
  status       ENUM('draft','published','cancelled','completed') NOT NULL DEFAULT
'draft',
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  PRIMARY KEY (event_id),
  KEY idx_events_start (start_time),
  KEY idx_events_status (status),
  KEY idx_events_sport (sport),
  KEY idx_events_venue (venue_id, start_time),
  KEY idx_events_organiser (organiser_id),
```

```
  CONSTRAINT fk_events_organiser FOREIGN KEY (organiser_id) REFERENCES users
(user_id),
  CONSTRAINT fk_events_venue     FOREIGN KEY (venue_id)     REFERENCES venues
(venue_id),
  CONSTRAINT chk_event_time CHECK (end_time > start_time)
) ENGINE=InnoDB

CREATE TABLE ticket_types (
  ticket_type_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  event_id       INT UNSIGNED NOT NULL,
  type_name      VARCHAR(50)  NOT NULL,
  price          DECIMAL(8,2) NOT NULL,
  capacity       INT UNSIGNED NOT NULL,
  sold           INT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (ticket_type_id),
  UNIQUE KEY uq_ticket_type (event_id, type_name),
  CONSTRAINT fk_ticket_event FOREIGN KEY (event_id) REFERENCES events (event_id)
ON DELETE CASCADE,
  CONSTRAINT chk_ticket_price CHECK (price >= 0),
  CONSTRAINT chk_ticket_sold  CHECK (sold <= capacity)
) ENGINE=InnoDB

CREATE TABLE bookings (
  booking_id     INT UNSIGNED NOT NULL AUTO_INCREMENT,
  booking_ref    CHAR(10)     NOT NULL,
  user_id        INT UNSIGNED NOT NULL,
  booking_type   ENUM('event_ticket','court') NOT NULL,
  ticket_type_id INT UNSIGNED NULL,
  court_id       INT UNSIGNED NULL,
  quantity       INT UNSIGNED NOT NULL DEFAULT 1,
  slot_start     DATETIME NULL,
  slot_end       DATETIME NULL,
  total_amount   DECIMAL(10,2) NOT NULL,
  status         ENUM('pending','confirmed','cancelled','completed') NOT NULL
DEFAULT 'pending',
  qr_code        MEDIUMTEXT NULL,
  created_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at     TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
  PRIMARY KEY (booking_id),
  UNIQUE KEY uq_booking_ref (booking_ref),
  KEY idx_bookings_user (user_id, created_at),
  KEY idx_bookings_status (status),
  KEY idx_bookings_ticket (ticket_type_id),
  KEY idx_bookings_court_slot (court_id, slot_start, slot_end),
```

```
  CONSTRAINT fk_booking_user   FOREIGN KEY (user_id)        REFERENCES users
(user_id),
  CONSTRAINT fk_booking_ticket FOREIGN KEY (ticket_type_id) REFERENCES
ticket_types (ticket_type_id),
  CONSTRAINT fk_booking_court  FOREIGN KEY (court_id)       REFERENCES courts
(court_id),
  CONSTRAINT chk_booking_qty   CHECK (quantity > 0)
) ENGINE=InnoDB

CREATE TABLE payments (
  payment_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
  booking_id INT UNSIGNED NOT NULL,
  amount     DECIMAL(10,2) NOT NULL,
  method     ENUM('card','fpx','ewallet','cash') NOT NULL,
  status     ENUM('pending','paid','failed','refunded') NOT NULL DEFAULT
'pending',
  paid_at    DATETIME NULL,
  PRIMARY KEY (payment_id),
  UNIQUE KEY uq_payment_booking (booking_id),
  CONSTRAINT fk_payment_booking FOREIGN KEY (booking_id) REFERENCES bookings
(booking_id) ON DELETE CASCADE
) ENGINE=InnoDB

CREATE TABLE audit_logs (
  log_id     BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id    INT UNSIGNED NULL,
  action     VARCHAR(60)  NOT NULL,
  detail     VARCHAR(255) NULL,
  ip_address VARCHAR(45)  NULL,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (log_id),
  KEY idx_audit_user (user_id),
  KEY idx_audit_action (action),
  CONSTRAINT fk_audit_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON
DELETE SET NULL
) ENGINE=InnoDB

CREATE TABLE request_logs (
  request_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id     INT UNSIGNED NULL,
  method      VARCHAR(10)  NOT NULL,
  path        VARCHAR(255) NOT NULL,
  status_code SMALLINT UNSIGNED NOT NULL,
  duration_ms INT UNSIGNED NOT NULL,
  ip_address  VARCHAR(45)  NULL,
```

```
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (request_id),
  KEY idx_request_created (created_at),
  KEY idx_request_status (status_code)
) ENGINE=InnoDB

CREATE TABLE rate_limits (
  bucket       VARCHAR(80)  NOT NULL,
  window_start INT UNSIGNED NOT NULL,
  hits         INT UNSIGNED NOT NULL DEFAULT 1,
  PRIMARY KEY (bucket, window_start)
) ENGINE=InnoDB
```

2.4 DATA MANIPULATION LANGUAGE (DML)

The DML below populates every table. Event and booking dates are generated relative to CURDATE() so the demo data never expires. All demo users share the password Password123! (bcrypt hash).

```
SQL
INSERT INTO users (user_id, role, full_name, email, password_hash, phone,
is_active) VALUES
 (1,'admin',    'Hafiq Admin',    'admin@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0111111111',1),
 (2,'organiser','Akma Organiser', 'organiser@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0122222222',1),
 (3,'organiser','Putera Sports',  'putera@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0133333333',1),
 (4,'customer', 'Arep Customer',  'customer@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0144444444',1),
 (5,'customer', 'Siti Aminah',    'siti@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0155555555',1),
 (6,'customer', 'Lim Wei',        'lim@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0166666666',1),
 (7,'customer', 'Nurul Huda',     'nurul@boom.test',
'$2y$10$Yn7TG/1gFvConvtYeMRoXeNFFrUHXLUHs6if.YHYGP6ivwGAa0Iou','0177777777',0)

INSERT INTO password_resets (reset_id, user_id, token_hash, expires_at, used_at)
VALUES
 (1,4,SHA2('demo-token-1',256),DATE_ADD(NOW(),INTERVAL 1 HOUR),NULL),
 (2,5,SHA2('demo-token-2',256),DATE_ADD(NOW(),INTERVAL 1 HOUR),NULL),
 (3,6,SHA2('demo-token-3',256),DATE_SUB(NOW(),INTERVAL 1 DAY),NULL),
```

```
 (4,3,SHA2('demo-token-4',256),DATE_SUB(NOW(),INTERVAL 2
DAY),DATE_SUB(NOW(),INTERVAL 2 DAY)),
 (5,2,SHA2('demo-token-5',256),DATE_ADD(NOW(),INTERVAL 30 MINUTE),NULL)

INSERT INTO venues (venue_id, owner_id, venue_name, address, city, capacity,
contact_phone, is_active) VALUES
 (1,2,'BOOM Arena Cheras',    '12 Jalan Cheras Sport', 'Kuala
Lumpur',500,'03-11111111',1),
 (2,2,'Pickle Point Bangsar', '88 Jalan Telawi',       'Kuala
Lumpur',200,'03-22222222',1),
 (3,3,'Shuttle Hub Shah Alam','5 Persiaran Sukan',     'Shah Alam',
300,'03-33333333',1),
 (4,3,'Futsal Zone Subang',   '21 Jalan SS15',         'Subang Jaya',
250,'03-44444444',1),
 (5,2,'Raja Court Penang',    '9 Lebuh Sukan',         'George Town',
400,'04-55555555',1)

INSERT INTO courts (court_id, venue_id, sport, court_name, hourly_rate,
is_active) VALUES
 (1,1,'Futsal',    'Futsal Court A',     120.00,1),
 (2,1,'Futsal',    'Futsal Court B',     110.00,1),
 (3,2,'Pickleball','Pickleball Court 1',  45.00,1),
 (4,2,'Pickleball','Pickleball Court 2',  45.00,1),
 (5,3,'Badminton', 'Badminton Court 1',   30.00,1),
 (6,3,'Badminton', 'Badminton Court 2',   30.00,1),
 (7,4,'Futsal',    'Futsal Court C',     105.00,1),
 (8,5,'Badminton', 'Badminton Court 3',   28.00,1)

INSERT INTO events (event_id, organiser_id, venue_id, sport, title, description,
start_time, end_time, status) VALUES
 (1,2,1,'Futsal',    'BOOM Futsal Cup',           '8-team knockout futsal
tournament',      TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 14
DAY),'09:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 14
DAY),'18:00:00'),'published'),
 (2,2,2,'Pickleball','Pickleball Open Day',       'Beginner-friendly pickleball
clinic',    TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 21
DAY),'10:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 21
DAY),'15:00:00'),'published'),
 (3,3,3,'Badminton', 'Smash Badminton Series',    'Doubles badminton league
night',         TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 30
DAY),'18:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 30
DAY),'22:00:00'),'published'),
 (4,3,4,'Futsal',    'Subang Futsal League',      'Weekly league fixtures',
TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 35
```

```
DAY),'19:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 35
DAY),'22:00:00'),'draft'),
 (5,2,5,'Badminton', 'Penang Badminton Masters',  'Open singles championship',
TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 60
DAY),'09:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 60
DAY),'17:00:00'),'published'),
 (6,2,1,'Futsal',    'Cheras Futsal Kids Clinic', 'Skills clinic for ages 8 to
12',         TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 45
DAY),'09:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 45
DAY),'12:00:00'),'published'),
 (7,3,2,'Pickleball','Pickle Mixed Doubles',      'Social mixed doubles round
robin',       TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 28
DAY),'14:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 28
DAY),'18:00:00'),'published'),
 (8,2,1,'Futsal',    'Legends Charity Match',     'Charity match with former
internationals',TIMESTAMP(DATE_SUB(CURDATE(),INTERVAL 20
DAY),'16:00:00'),TIMESTAMP(DATE_SUB(CURDATE(),INTERVAL 20
DAY),'19:00:00'),'completed')

INSERT INTO ticket_types (ticket_type_id, event_id, type_name, price, capacity,
sold) VALUES
 (1, 1,'General',25.00,400,2),
 (2, 1,'VIP',    60.00, 50,0),
 (3, 2,'General',15.00,150,1),
 (4, 3,'VIP',    40.00, 50,1),
 (5, 3,'General',20.00,200,2),
 (6, 4,'General',20.00,200,0),
 (7, 5,'General',18.00,300,0),
 (8, 6,'General',10.00, 60,0),
 (9, 7,'General',12.00, 80,0),
 (10,8,'General',15.00,300,3)

INSERT INTO bookings (booking_id, booking_ref, user_id, booking_type,
ticket_type_id, court_id, quantity, slot_start, slot_end, total_amount, status)
VALUES
 (1,'BM00000001',4,'event_ticket',1,   NULL,2,NULL,NULL,50.00,'confirmed'),
 (2,'BM00000002',5,'event_ticket',3,   NULL,1,NULL,NULL,15.00,'confirmed'),
 (3,'BM00000003',6,'event_ticket',4,   NULL,1,NULL,NULL,40.00,'pending'),
 (4,'BM00000004',4,'court',      NULL, 5,
1,TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 3
DAY),'20:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 3
DAY),'21:00:00'),30.00,'confirmed'),
 (5,'BM00000005',5,'event_ticket',7,   NULL,1,NULL,NULL,18.00,'cancelled'),
 (6,'BM00000006',4,'event_ticket',5,   NULL,2,NULL,NULL,40.00,'confirmed'),
 (7,'BM00000007',6,'event_ticket',10,  NULL,3,NULL,NULL,45.00,'completed'),
```

```
 (8,'BM00000008',5,'court',      NULL, 3,
1,TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 5
DAY),'19:00:00'),TIMESTAMP(DATE_ADD(CURDATE(),INTERVAL 5
DAY),'21:00:00'),90.00,'confirmed')

INSERT INTO payments (payment_id, booking_id, amount, method, status, paid_at)
VALUES
 (1,1,50.00,'card',   'paid',    NOW()),
 (2,2,15.00,'fpx',    'paid',    NOW()),
 (3,3,40.00,'cash',   'pending', NULL),
 (4,4,30.00,'ewallet','paid',    NOW()),
 (5,5,18.00,'card',   'refunded',NOW()),
 (6,6,40.00,'fpx',    'paid',    NOW()),
 (7,7,45.00,'card',   'paid',    DATE_SUB(NOW(),INTERVAL 21 DAY)),
 (8,8,90.00,'card',   'paid',    NOW())

INSERT INTO audit_logs (user_id, action, detail, ip_address) VALUES
 (1,'LOGIN','Admin login','127.0.0.1'),
 (2,'EVENT_CREATE','Created BOOM Futsal Cup','127.0.0.1'),
 (4,'REGISTER','New customer registered','127.0.0.1'),
 (4,'BOOKING_CREATE','Booking BM00000001','127.0.0.1'),
 (5,'BOOKING_CANCEL','Booking BM00000005','127.0.0.1')

INSERT INTO request_logs (user_id, method, path, status_code, duration_ms,
ip_address) VALUES
 (NULL,'GET', '/events',          200, 12,'127.0.0.1'),
 (NULL,'POST','/auth/login',      401, 85,'127.0.0.1'),
 (4,   'POST','/bookings',        201, 48,'127.0.0.1'),
 (2,   'PUT', '/events/1',        200, 21,'127.0.0.1'),
 (NULL,'GET', '/unknown',         404,  3,'127.0.0.1')

INSERT INTO rate_limits (bucket, window_start, hits) VALUES
 ('ip:127.0.0.1',        1700000000, 12),
 ('ip:127.0.0.2',        1700000000,  4),
 ('login:127.0.0.1',     1700000000,  3),
 ('login:127.0.0.2',     1700000000,  1),
 ('register:127.0.0.1',  1700000000,  2)
```

Import: mysql -u root < database/boom_plus.sql or phpMyAdmin → Import.

3. RESTful API DOCUMENTATION

Base URL: http://localhost/boom/api. All requests and responses use JSON. Protected endpoints need the header Authorization: Bearer <JWT> obtained from POST /auth/login. Responses use this envelope:

```
None
Success: { "success": true, "message": "...", "data": ..., "meta": { "total": 8,
"page": 1, "limit": 10, "pages": 1 } }
Error:   { "success": false, "message": "...", "errors": { "field": "reason" } }
```

List query parameters: page, limit (default 10, maximum 100), sort (comma list, prefix - for descending), q (search) and resource-specific filters. **Roles:** Public = no token Any user = valid token Customer, Organiser, Admin = that role (Admin passes every organiser check). Sample responses are abridged.

Status	Meaning
200 / 201	OK / created
400	Malformed JSON or business rule (for example wrong current password)
401	Missing, invalid or expired token disabled account
403	Authenticated but role or ownership is not allowed
404	Resource not found (or hidden from the role)
405	Method not allowed
409	Conflict: sold out, slot taken, schedule clash, duplicate, record in use
422	Validation failed (errors lists each field)
429	Rate limit exceeded (Retry-After header)
500	Unexpected error (details only in the server log)

Table 3.1: Standard API HTTP Status Codes and Descriptions

3.1 Authentication

3.1.1 Register

Purpose	Creates a customer account. The role is always the customer.
URL	/auth/register
HTTP method	POST
Authentication / role	Public

Request parameters / body	full_name (required, max 100), email (required, unique), password (required, strong), phone (optional) { "full_name": "Aina Rahman", "email": "aina@example.com", "password": "Password123!", "phone": "0123456789" }
Sample JSON response	{ "success": true, "message": "Registered", "data": { "user_id": 8 } }
HTTP status codes	201 Created 409 email already registered 422 validation failed 429 rate limit

Table 3.2: API Endpoint Specification – POST /auth/register

3.1.2 Login

Purpose	Verifies credentials and returns a signed JWT (HS256, 3600 seconds) with the user profile.
URL	/auth/login
HTTP method	POST
Authentication / role	Public
Request parameters / body	email (required), password (required) { "email": "customer@boom.test", "password": "Password123!" }
Sample JSON response	{ "success": true, "data": { "token": "eyJhbGciOiJIUzI1NiIs...", "token_type": "Bearer", "expires_in": 3600, "user": { "user_id": 4, "full_name": "Arep Customer", "email": "customer@boom.test", "role": "customer" } } }
HTTP status codes	200 OK 401 invalid email or password 403 account disabled 422 validation failed 429 rate limit

Table 3.3: API Endpoint Specification – POST /auth/login

3.1.3 Current user

Purpose	Returns the profile of the token owner (re-read from the database).
URL	/auth/me
HTTP method	GET
Authentication / role	Any user
Request parameters / body	None (header Authorization: Bearer <token>)
Sample JSON response	{ "success": true, "data": { "user_id": 4, "full_name": "Arep Customer", "email": "customer@boom.test", "phone": "0144444444", "is_active": 1, "role": "customer" } }
HTTP status codes	200 OK 401 missing or invalid token

Table 3.4: API Endpoint Specification – GET /auth/me

3.1.4 Forgot password

Purpose	Creates a single-use reset token (valid 1 hour). The response is the same whether or not the email exists. In development the token is returned as dev_reset_token.
URL	/auth/forgot-password
HTTP method	POST
Authentication / role	Public
Request parameters / body	email (required)
Sample JSON response	{ "success": true, "message": "If that email exists, a reset link has been sent", "dev_reset_token": "9f2c...e1" }
HTTP status codes	200 OK 422 validation failed 429 rate limit

Table 3.5: API Endpoint Specification – POST /auth/forgot-password

3.1.5 Reset password

Purpose	Sets a new password using a valid, unused, unexpired reset token.
URL	/auth/reset-password
HTTP method	POST
Authentication / role	Public
Request parameters / body	token (required), password (required, strong)
Sample JSON response	{ "success": true, "message": "Password updated", "data": null }
HTTP status codes	200 OK 400 invalid or expired token 422 validation failed 429 rate limit

Table 3.6: API Endpoint Specification – POST /auth/reset-password

3.2 Users

3.2.1 Get own profile

Purpose	Returns the logged-in user's profile.
URL	/users/me
HTTP method	GET

Authentication / role	Any user
Request parameters / body	None
Sample JSON response	{ "success": true, "data": { "user_id": 4, "full_name": "Arep Customer", "email": "customer@boom.test", "phone": "0144444444", "is_active": 1, "role": "customer" } }
HTTP status codes	200 OK 401

Table 3.7: API Endpoint Specification – GET /users/me

3.2.2 Update own profile

Purpose	Updates name and phone of the logged-in user.
URL	/users/me
HTTP method	PUT
Authentication / role	Any user
Request parameters / body	full_name (optional, max 100), phone (optional) { "full_name": "Arep Ahmad", "phone": "0144444444" }
Sample JSON response	{ "success": true, "message": "Profile updated", "data": null }
HTTP status codes	200 OK 401 422

Table 3.8: API Endpoint Specification – PUT /users/me

3.2.3 Change password

Purpose	Changes the password after checking the current one.
URL	/users/me/password
HTTP method	PUT
Authentication / role	Any user
Request parameters / body	current_password (required), new_password (required, strong)
Sample JSON response	{ "success": true, "message": "Password changed", "data": null }
HTTP status codes	200 OK 400 current password incorrect 401 422

Table 3.9: API Endpoint Specification – PUT /users/me/password

3.2.4 List users

Purpose	Lists users with filters and pagination.
URL	/users
HTTP method	GET
Authentication / role	Admin
Request parameters / body	Query: role, is_active, q, page, limit, sort (id, name, email, role, created_at)
Sample JSON response	{ "success": true, "data": [{ "user_id": 4, "full_name": "Arep Customer", "email": "customer@boom.test", "phone": "0144444444", "is_active": 1, "role": "customer" }], "meta": { "total": 7, "page": 1, "limit": 10, "pages": 1 } }
HTTP status codes	200 OK 401 403 not admin

Table 3.10: API Endpoint Specification – GET /users

3.2.5 Create user

Purpose	Creates a user with any role.
URL	/users
HTTP method	POST
Authentication / role	Admin
Request parameters / body	full_name, email, password, role (admin | organiser | customer) required phone optional
Sample JSON response	{ "success": true, "message": "User created", "data": { "user_id": 9 } }
HTTP status codes	201 Created 401 403 409 email exists 422

Table 3.11: API Endpoint Specification – POST /users

3.2.6 Get user

Purpose	Returns one user.
URL	/users/{id}
HTTP method	GET
Authentication / role	Admin
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": { "user_id": 4, "full_name": "Arep Customer", "email": "customer@boom.test", "phone": "0144444444", "is_active": 1, "role": "customer" } }

HTTP status codes	200 OK 401 403 404

Table 3.12: API Endpoint Specification – GET /users/{id}

3.2.7 Change role

Purpose	Changes the role of a user. An administrator cannot change their own role.
URL	/users/{id}/role
HTTP method	PUT
Authentication / role	Admin
Request parameters / body	role (required: admin | organiser | customer) { "role": "organiser" }
Sample JSON response	{ "success": true, "message": "Role updated", "data": null }
HTTP status codes	200 OK 400 own role 401 403 404 422

Table 3.13: API Endpoint Specification – PUT /users/{id}/role

3.2.8 Enable or disable user

Purpose	Enables or disables an account (disabled users cannot log in or use a token).
URL	/users/{id}/status
HTTP method	PUT
Authentication / role	Admin
Request parameters / body	is_active (required: 0 or 1)
Sample JSON response	{ "success": true, "message": "Status updated", "data": null }
HTTP status codes	200 OK 400 cannot disable yourself 401 403 404 422

Table 3.14: API Endpoint Specification – PUT /users/{id}/status

3.2.9 Delete user

Purpose	Deletes a user who owns no data.
URL	/users/{id}
HTTP method	DELETE
Authentication / role	Admin
Request parameters / body	Path: id

Sample JSON response	{ "success": true, "message": "User deleted", "data": null }
HTTP status codes	200 OK 400 cannot delete yourself 401 403 404 409 user has venues, events or bookings

Table 3.15: API Endpoint Specification – DELETE /users/{id}

3.3 Venues and Courts

3.3.1 List venues

Purpose	Lists venues with court count and sports.
URL	/venues
HTTP method	GET
Authentication / role	Public
Request parameters / body	Query: city, sport, q, mine=1 (organiser), is_active, sort (name, city, capacity, created_at), page, limit
Sample JSON response	{ "success": true, "data": [{ "venue_id": 1, "owner_id": 2, "venue_name": "BOOM Arena Cheras", "address": "12 Jalan Cheras Sport", "city": "Kuala Lumpur", "capacity": 500, "contact_phone": "03-11111111", "is_active": 1, "owner_name": "Akma Organiser", "court_count": 2, "sports": "Futsal" }], "meta": { "total": 5, "page": 1, "limit": 10, "pages": 1 } }
HTTP status codes	200 OK 401 invalid token 422 invalid filter

Table 3.16: API Endpoint Specification – GET /venues

3.3.2 Create venue

Purpose	Creates a venue owned by the caller (admin may set owner_id).
URL	/venues
HTTP method	POST
Authentication / role	Organiser, Admin
Request parameters / body	venue_name, address, city, capacity (required) contact_phone, is_active, owner_id (admin) optional { "venue_name": "Test Court Cheras", "address": "1 Jalan Sukan", "city": "Kuala Lumpur", "capacity": 120 }
Sample JSON response	{ "success": true, "message": "Venue created", "data": { "venue_id": 6, "venue_name": "Test Court Cheras", "capacity": 120 } }
HTTP status codes	201 Created 401 403 422

Table 3.17: API Endpoint Specification – POST /venues

3.3.3 Get venue

Purpose	Returns one venue including its courts.
URL	/venues/{id}
HTTP method	GET
Authentication / role	Public
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": { "venue_id": 1, "venue_name": "BOOM Arena Cheras", "capacity": 500, "courts": [{ "court_id": 1, "sport": "Futsal", "court_name": "Futsal Court A", "hourly_rate": 120.00 }] } }
HTTP status codes	200 OK 404

Table 3.18: API Endpoint Specification – GET /venues/{id}

3.3.4 Update venue

Purpose	PUT replaces all fields PATCH changes only the fields sent (for example is_active).
URL	/venues/{id}
HTTP method	PUT / PATCH
Authentication / role	Owner, Admin
Request parameters / body	Same fields as create PATCH accepts any subset
Sample JSON response	{ "success": true, "message": "Venue updated", "data": { "venue_id": 1, "is_active": 0 } }
HTTP status codes	200 OK 401 403 not owner 404 422

Table 3.19: API Endpoint Specification – PUT/PATCH /venues/{id}

3.3.5 Delete venue

Purpose	Deletes a venue that has no events or bookings.
URL	/venues/{id}
HTTP method	DELETE
Authentication / role	Owner, Admin
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "message": "Venue deleted", "data": null }

HTTP status codes	200 OK 401 403 404 409 events or bookings exist

Table 3.20: API Endpoint Specification – DELETE /venues/{id}

3.3.6 List courts

Purpose	Lists courts of a venue.
URL	/venues/{id}/courts
HTTP method	GET
Authentication / role	Public
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": [{ "court_id": 1, "venue_id": 1, "sport": "Futsal", "court_name": "Futsal Court A", "hourly_rate": 120.00, "is_active": 1 }] }
HTTP status codes	200 OK 404

Table 3.21: API Endpoint Specification – GET /venues/{id}/courts

3.3.7 Add court

Purpose	Adds a court to a venue.
URL	/venues/{id}/courts
HTTP method	POST
Authentication / role	Owner, Admin
Request parameters / body	sport (Pickleball | Futsal | Badminton), court_name, hourly_rate required is_active optional { "sport": "Futsal", "court_name": "Futsal Court D", "hourly_rate": 115 }
Sample JSON response	{ "success": true, "message": "Court created", "data": { "court_id": 9, "court_name": "Futsal Court D" } }
HTTP status codes	201 Created 401 403 404 409 duplicate name 422

Table 3.22: API Endpoint Specification – POST /venues/{id}/courts

3.3.8Update court

Purpose	Updates a court.
URL	/venues/{id}/courts/{courtId}
HTTP method	PUT / PATCH
Authentication / role	Owner, Admin
Request parameters / body	Same fields as add court
Sample JSON response	{ "success": true, "message": "Court updated", "data": { "court_id": 9, "hourly_rate": 118.00 } }
HTTP status codes	200 OK 401 403 404 409 422

Table 3.23: API Endpoint Specification – PUT/PATCH /venues/{id}/courts/{courtId}

3.3.9Delete court

Purpose	Deletes a court that has no bookings.
URL	/venues/{id}/courts/{courtId}
HTTP method	DELETE
Authentication / role	Owner, Admin
Request parameters / body	Path: id, courtId
Sample JSON response	{ "success": true, "message": "Court deleted", "data": null }

HTTP status codes	200 OK 401 403 404 409 in use

Table 3.24: API Endpoint Specification – DELETE /venues/{id}/courts/{courtId}

3.3.10Venue schedule

Purpose	Returns events and booked court slots for one day.
URL	/venues/{id}/schedule
HTTP method	GET
Authentication / role	Public
Request parameters / body	Query: date=YYYY-MM-DD (default today)
Sample JSON response	{ "success": true, "data": { "venue_id": 1, "venue": "BOOM Arena Cheras", "date": "2030-01-15", "events": [], "courts": [{ "court_id": 1, "court_name": "Futsal Court A", "booked_slots": [] }] } }
HTTP status codes	200 OK 404 422 invalid date

Table 3.25: API Endpoint Specification – GET /venues/{id}/schedule

3.4 Events and Ticket Types

3.4.1List events

Purpose	Lists events. Customers see published and completed events organisers also see their own drafts.
URL	/events
HTTP method	GET
Authentication / role	Public
Request parameters / body	Query: sport, status, venue_id, city, from, to, upcoming=1, available=1, mine=1, q, sort (start_time, title, price, created_at, city), page, limit
Sample JSON response	{ "success": true, "data": [{ "event_id": 1, "organiser_id": 2, "venue_id": 1, "sport": "Futsal", "title": "BOOM Futsal Cup", "start_time": "2026-10-22 09:00:00", "end_time": "2026-10-22 18:00:00", "status": "published", "venue_name": "BOOM Arena Cheras", "city": "Kuala Lumpur", "min_price": 25.00, "total_capacity": 450, "tickets_left": 448 }], "meta": { "total": 7, "page": 1, "limit": 10, "pages": 1 } }
HTTP status codes	200 OK 401 invalid token 422 invalid filter

Table 3.26: API Endpoint Specification – GET /events

3.4.2Create event

Purpose	Creates an event and its ticket types in one transaction.
URL	/events
HTTP method	POST
Authentication / role	Organiser, Admin
Request parameters / body	venue_id, sport, title (min 3), start_time, end_time required (start in the future, end after start) description, status (draft | published), ticket_types[] optional { "venue_id": 1, "sport": "Futsal", "title": "Test Futsal Night", "start_time": "2030-01-15 19:00:00", "end_time": "2030-01-15 22:00:00", "status": "published", "ticket_types": [{ "type_name": "General", "price": 20, "capacity": 100 }] }
Sample JSON response	{ "success": true, "message": "Event created", "data": { "event_id": 9, "title": "Test Futsal Night", "status": "published", "ticket_types": [{ "ticket_type_id": 11, "type_name": "General", "price": 20.00, "capacity": 100, "sold": 0, "remaining": 100 }] } }
HTTP status codes	201 Created 401 403 404 venue 409 overlapping event at the venue 422 validation or capacity above venue capacity

Table 3.27: API Endpoint Specification – POST /events

3.4.3Get event

Purpose	Returns one event with ticket types and remaining stock.
URL	/events/{id}
HTTP method	GET
Authentication / role	Public (drafts: owner, Admin)
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": { "event_id": 1, "title": "BOOM Futsal Cup", "status": "published", "ticket_types": [{ "type_name": "General", "price": 25.00, "capacity": 400, "sold": 2, "remaining": 398 }, { "type_name": "VIP", "price": 60.00, "capacity": 50, "sold": 0, "remaining": 50 }] } }
HTTP status codes	200 OK 404 not found or hidden draft

Table 3.28: API Endpoint Specification – GET /events/{id}

3.4.4Update event

Purpose	Updates an event. Setting status cancelled cancels and refunds all active bookings.
URL	/events/{id}

HTTP method	PUT / PATCH
Authentication / role	Owner, Admin
Request parameters / body	Same fields as create PATCH accepts any subset, for example { "status": "cancelled" }
Sample JSON response	{ "success": true, "message": "Event updated", "data": { "event_id": 1, "status": "cancelled" } }
HTTP status codes	200 OK 401 403 404 409 clash 422

Table 3.29: API Endpoint Specification – PUT/PATCH /events/{id}

3.4.5Delete event

Purpose	Deletes an event that has no bookings.
URL	/events/{id}
HTTP method	DELETE
Authentication / role	Owner, Admin
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "message": "Event deleted", "data": null }
HTTP status codes	200 OK 401 403 404 409 bookings exist

Table 3.30: API Endpoint Specification – DELETE /events/{id}

3.4.6List ticket types

Purpose	Lists ticket types with remaining stock.
URL	/events/{id}/tickets
HTTP method	GET
Authentication / role	Public
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": [{ "ticket_type_id": 1, "type_name": "General", "price": 25.00, "capacity": 400, "sold": 2, "remaining": 398 }] }
HTTP status codes	200 OK 404

Table 3.31: API Endpoint Specification – GET /events/{id}/tickets

3.4.7Add ticket type

Purpose	Adds a ticket type total capacity cannot exceed the venue capacity.
URL	/events/{id}/tickets
HTTP method	POST
Authentication / role	Owner, Admin
Request parameters / body	type_name, price, capacity required { "type_name": "Student", "price": 12, "capacity": 50 }
Sample JSON response	{ "success": true, "message": "Ticket type created", "data": { "ticket_type_id": 12, "type_name": "Student" } }
HTTP status codes	201 Created 401 403 404 409 duplicate 422

Table 3.32: API Endpoint Specification – POST /events/{id}/tickets

3.4.8Update ticket type

Purpose	Updates price or capacity. Capacity cannot be lower than tickets sold.
URL	/events/{id}/tickets/{ticketId}
HTTP method	PUT / PATCH
Authentication / role	Owner, Admin
Request parameters / body	type_name, price, capacity
Sample JSON response	{ "success": true, "message": "Ticket type updated", "data": { "ticket_type_id": 12, "capacity": 60 } }
HTTP status codes	200 OK 401 403 404 409 422

Table 3.33: API Endpoint Specification – PUT/PATCH /events/{id}/tickets/{ticketId}

3.4.9 Delete ticket type

Purpose	Deletes a ticket type that has no bookings.
URL	/events/{id}/tickets/{ticketId}
HTTP method	DELETE
Authentication / role	Owner, Admin
Request parameters / body	Path: id, ticketId
Sample JSON response	{ "success": true, "message": "Ticket type deleted", "data": null }
HTTP status codes	200 OK 401 403 404 409 in use

Table 3.34: API Endpoint Specification – DELETE /events/{id}/tickets/{ticketId}

3.4.10 Event availability

Purpose	Returns real-time seats left per ticket type and a bookable flag.
URL	/events/{id}/availability
HTTP method	GET
Authentication / role	Public
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": { "event_id": 1, "title": "BOOM Futsal Cup", "bookable": true, "tickets_left": 448, "ticket_types": [{ "ticket_type_id": 1, "type_name": "General", "remaining": 398 }] } }
HTTP status codes	200 OK 404

Table 3.35: API Endpoint Specification – GET /events/{id}/availability

3.5 Bookings and Payments

3.5.1 List bookings

Purpose	Customers see their own bookings, organisers see bookings for their own events and venues, administrators see all.
URL	/bookings
HTTP method	GET
Authentication / role	Any user (scoped)
Request parameters / body	Query: status, booking_type, event_id, user_id (staff), from, to, view=active|history, q, sort (created_at, amount, status, starts_at, ref), page, limit
Sample JSON response	{ "success": true, "data": [{ "booking_id": 1, "booking_ref": "BM00000001", "user_id": 4, "booking_type": "event_ticket", "quantity": 2, "total_amount": 50.00, "status": "confirmed", "item_title": "BOOM Futsal Cup", "venue_name": "BOOM Arena Cheras", "payment_method": "card", "payment_status": "paid" }], "meta": { "total": 4, "page": 1, "limit": 10, "pages": 1 } }
HTTP status codes	200 OK 401 422 invalid filter

Table 3.36: API Endpoint Specification – GET /bookings

3.5.2 Create booking

Purpose	Books event tickets or a court slot and records the payment. Stock is locked with SELECT ... FOR UPDATE in a transaction. Card, FPX and e-wallet are confirmed immediately, cash stays pending.
URL	/bookings
HTTP method	POST
Authentication / role	Customer
Request parameters / body	payment_method (card | fpx | ewallet | cash) and booking_type required. event_ticket: ticket_type_id, quantity (1 to 10). court: court_id, slot_start, slot_end (whole hours, 1 to 4 hours). { "booking_type": "event_ticket", "ticket_type_id": 1, "quantity": 2, "payment_method": "ewallet" }
Sample JSON response	{ "success": true, "message": "Booking created", "data": { "booking_id": 9, "booking_ref": "BM3FA1C20D", "quantity": 2, "total_amount": 50.00, "status": "confirmed", "payment_status": "paid", "qr_code": "data:image/pngbase64,iVBOR..." } }
HTTP status codes	201 Created 401 403 not a customer 404 ticket or court 409 sold out, event closed or started, slot taken 422 validation failed

Table 3.37: API Endpoint Specification – POST /bookings

3.5.3 Get booking

Purpose	Returns one booking including the QR code (created on first view if missing).
URL	/bookings/{id}
HTTP method	GET
Authentication / role	Owner, organiser of the event, Admin
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "data": { "booking_id": 1, "booking_ref": "BM00000001", "user_id": 4, "booking_type": "event_ticket", "quantity": 2, "total_amount": 50.00, "status": "confirmed", "item_title": "BOOM Futsal Cup", "venue_name": "BOOM Arena Cheras", "payment_method": "card", "payment_status": "paid", "qr_code": "data:image/pngbase64,iVBOR..." } }
HTTP status codes	200 OK 401 404 not found or not in scope

Table 3.38: API Endpoint Specification – GET /bookings/{id}

3.5.4 Modify booking

Purpose	Changes the ticket quantity or the court slot and re-validates availability and the price.
URL	/bookings/{id}
HTTP method	PUT / PATCH
Authentication / role	Owner (customer), Admin
Request parameters / body	event_ticket: quantity. court: slot_start, slot_end { "quantity": 3 }
Sample JSON response	{ "success": true, "message": "Booking updated", "data": { "booking_id": 9, "quantity": 3, "total_amount": 75.00 } }
HTTP status codes	200 OK 401 403 404 409 not enough stock, booking not modifiable or already started 422

Table 3.39: API Endpoint Specification – PUT/PATCH /bookings/{id}

3.5.5 Cancel booking

Purpose	Cancels a booking, releases the tickets and refunds a paid payment.
URL	/bookings/{id}/cancel or /bookings/{id}
HTTP method	POST (or DELETE)
Authentication / role	Owner, Admin
Request parameters / body	Path: id
Sample JSON response	{ "success": true, "message": "Booking cancelled", "data": { "booking_id": 9, "status": "cancelled", "payment_status": "refunded" } }
HTTP status codes	200 OK 401 403 404 409 booking already cancelled or completed, or already started

Table 3.40: API Endpoint Specification – POST /bookings/{id}/cancel

3.5.6 Change booking status

Purpose	Moves a booking through the workflow: pending → confirmed / cancelled, confirmed → completed / cancelled. Confirming marks a pending payment as paid.
URL	/bookings/{id}/status
HTTP method	PUT / PATCH
Authentication / role	Organiser, Admin
Request parameters / body	status (required: pending | confirmed | cancelled | completed) { "status": "completed" }
Sample JSON response	{ "success": true, "message": "Booking status updated", "data": { "booking_id": 1, "status": "completed" } }
HTTP status codes	200 OK 401 403 404 409 invalid transition 422

Table 3.41: API Endpoint Specification – PUT/PATCH /bookings/{id}/status

3.6 System, Statistics and Logs

3.6.1 Health check

Purpose	Checks that the API and database are running.
URL	/system/health
HTTP method	GET
Authentication / role	Public

Request parameters / body	None
Sample JSON response	{ "success": true, "data": { "status": "ok", "database": "up", "time": "2026-10-08T14:30:00+08:00" } }
HTTP status codes	200 OK 429

Table 3.42: API Endpoint Specification – GET /system/health

3.6.2Statistics

Purpose	Admin: users, venues, events, bookings, revenue and today's requests and errors. Organiser: own venues, events, tickets sold, pending bookings and revenue. Customer: active and total bookings, cancelled and total spent.
URL	/system/stats
HTTP method	GET
Authentication / role	Any user (role-specific)
Request parameters / body	None
Sample JSON response	{ "success": true, "data": { "users": 7, "venues": 5, "events": 8, "published_events": 6, "bookings": 8, "revenue": 288.00, "requests_today": 41, "errors_today": 0 } }
HTTP status codes	200 OK 401

Table 3.43: API Endpoint Specification – GET /system/stats

3.6.3Audit logs

Purpose	Lists audit trail records.
URL	/system/audit-logs
HTTP method	GET
Authentication / role	Admin
Request parameters / body	Query: action, user_id, q, sort (created_at, action), page, limit
Sample JSON response	{ "success": true, "data": [{ "log_id": 5, "user_id": 5, "user_name": "Siti Aminah", "action": "BOOKING_CANCEL", "detail": "Booking BM00000005", "ip_address": "127.0.0.1", "created_at": "2026-10-08 14:10:00" }], "meta": { "total": 5, "page": 1, "limit": 10, "pages": 1 } }
HTTP status codes	200 OK 401 403

Table 3.44: API Endpoint Specification – GET /system/audit-logs

3.6.4Request logs

Purpose	Lists every API request with its response time.
URL	/system/request-logs
HTTP method	GET
Authentication / role	Admin
Request parameters / body	Query: method, status_code, q, page, limit
Sample JSON response	{ "success": true, "data": [{ "request_id": 3, "user_id": 4, "method": "POST", "path": "/bookings", "status_code": 201, "duration_ms": 48, "ip_address": "127.0.0.1" }], "meta": { "total": 5, "page": 1, "limit": 10, "pages": 1 } }
HTTP status codes	200 OK 401 403

Table 3.45: API Endpoint Specification – GET /system/request-logs

4. API TESTING

4.1 TESTING SETUP

All endpoints are tested with **Postman**. The collection postman/BOOM_Plus.postman_collection.json and the environment postman/BOOM_Plus.postman_environment.json (variable baseUrl = http://localhost/boom/api) are exported and included in the repository. Import both files into Postman and select the **BOOM+ Local** environment. The three login requests save adminToken, organiserToken and customerToken automatically, and every request has test scripts that assert the status code and the response shape. Run the folders in this order: Auth, Users, Venues, Events, Bookings, Negative tests, System.

4.2 EXPORTED POSTMAN COLLECTION

The exported collection contains **29 requests** in 7 folders. It is submitted as postman/BOOM_Plus.postman_collection.json together with the environment file.

Folder	Method	URL	Request name
Auth	POST	{{baseUrl}}/auth/login	Login admin
Auth	POST	{{baseUrl}}/auth/login	Login organiser
Auth	POST	{{baseUrl}}/auth/login	Login customer
Auth	POST	{{baseUrl}}/auth/register	Register customer
Auth	GET	{{baseUrl}}/auth/me	Current user
Users	GET	{{baseUrl}}/users?limit=5&sort=name	List users (admin)
Users	PUT	{{baseUrl}}/users/me	Update my profile
Venues	GET	{{baseUrl}}/venues?sport=Futsal&sort=-capacity&limit=5	List venues
Venues	POST	{{baseUrl}}/venues	Create venue
Venues	POST	{{baseUrl}}/venues/{{venueId}}/courts	Add court
Venues	GET	{{baseUrl}}/venues/1/schedule?date=2030-01-15	Venue schedule
Venues	PATCH	{{baseUrl}}/venues/{{venueId}}	Update venue
Events	GET	{{baseUrl}}/events?sport=Futsal&sort=-start_time&limit=2	List events
Events	POST	{{baseUrl}}/events	Create event
Events	GET	{{baseUrl}}/events/{{eventId}}/availability	Availability

Folder	Method	URL	Request name
Events	POST	{{baseUrl}}/events	Overlapping event is rejected
Bookings	POST	{{baseUrl}}/bookings	Book ticket
Bookings	GET	{{baseUrl}}/bookings?view=active&sort=starts_at	My bookings
Bookings	PUT	{{baseUrl}}/bookings/{{bookingId}}	Modify quantity
Bookings	POST	{{baseUrl}}/bookings/{{bookingId}}/cancel	Cancel booking
Bookings	GET	{{baseUrl}}/bookings?limit=5	Organiser bookings
Negative tests	POST	{{baseUrl}}/auth/login	Wrong password
Negative tests	GET	{{baseUrl}}/users	Customer cannot list users
Negative tests	GET	{{baseUrl}}/bookings	No token
Negative tests	POST	{{baseUrl}}/bookings	Validation error
Negative tests	GET	{{baseUrl}}/events/99999	Unknown event
System	GET	{{baseUrl}}/system/health	Health
System	GET	{{baseUrl}}/system/stats	Stats (admin)
System	GET	{{baseUrl}}/system/request-logs?limit=5	Request logs (admin)

Table 4.1: Exported Postman Test Collection Endpoints and Methods

4.3 SUCCESSFUL RESPONSES

Request	Expected result	Figure
POST /auth/login (admin, organiser, customer)	200, token returned, token saved to environment	Figure 4.1
GET /venues?sport=Futsal&sort=-capacity&limit=5	200, meta.total present	Figure 4.2
POST /events	201, event with ticket types	Figure 4.3
GET /events?sport=Futsal&sort=-start_time&limit=2	200, filtered and sorted list	Figure 4.4
POST /bookings (event_ticket)	201, status confirmed, qr_code data URI	Figure 4.5
POST /bookings/{id}/cancel	200, status cancelled, seats released	Figure 4.6
GET /system/health	200, database up	Figure 4.7

Table 4.2: Postman Test Suite – Successful Response Test Cases

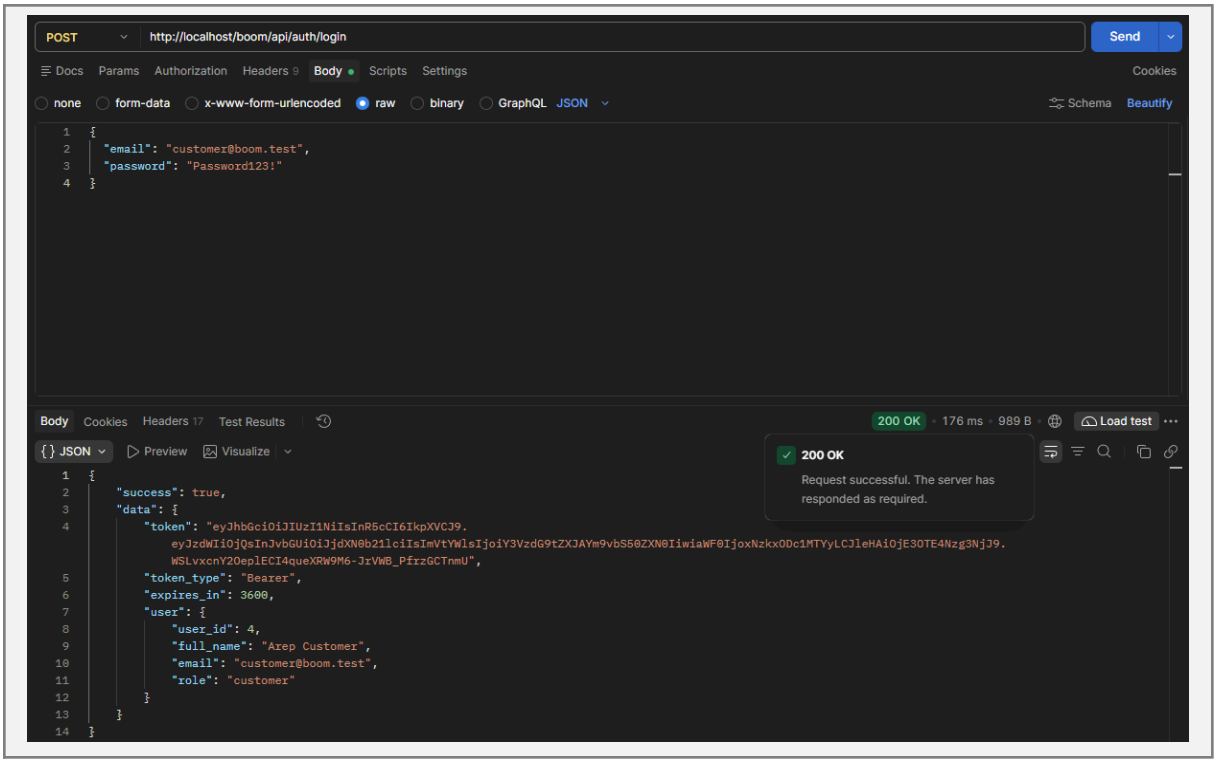


Figure 4.1: POST /auth/login (200 OK)

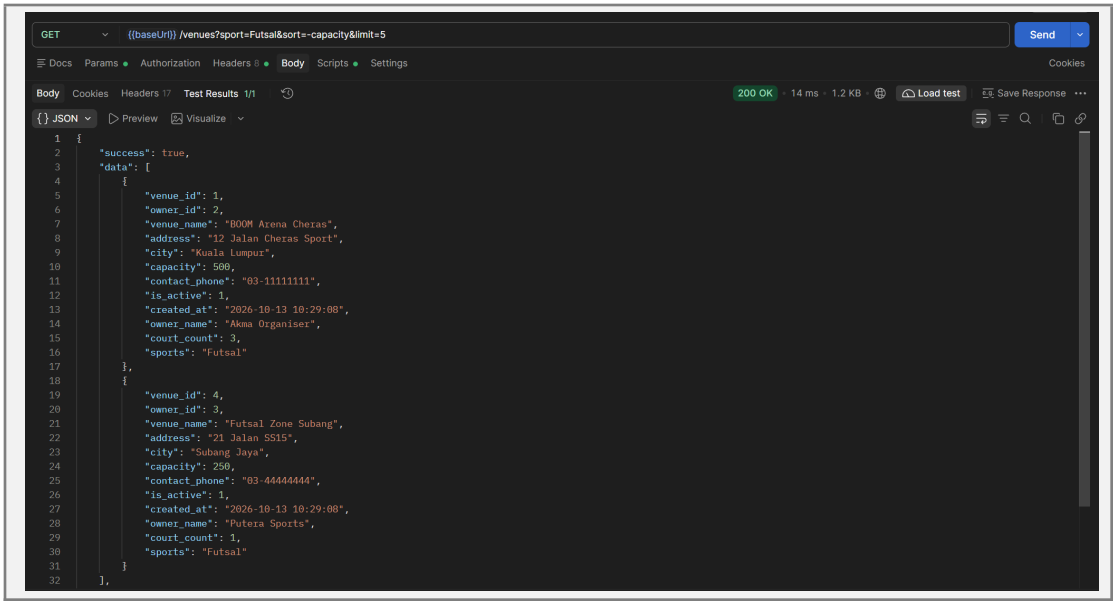


Figure 4.2: GET /venues with filters (200 OK)

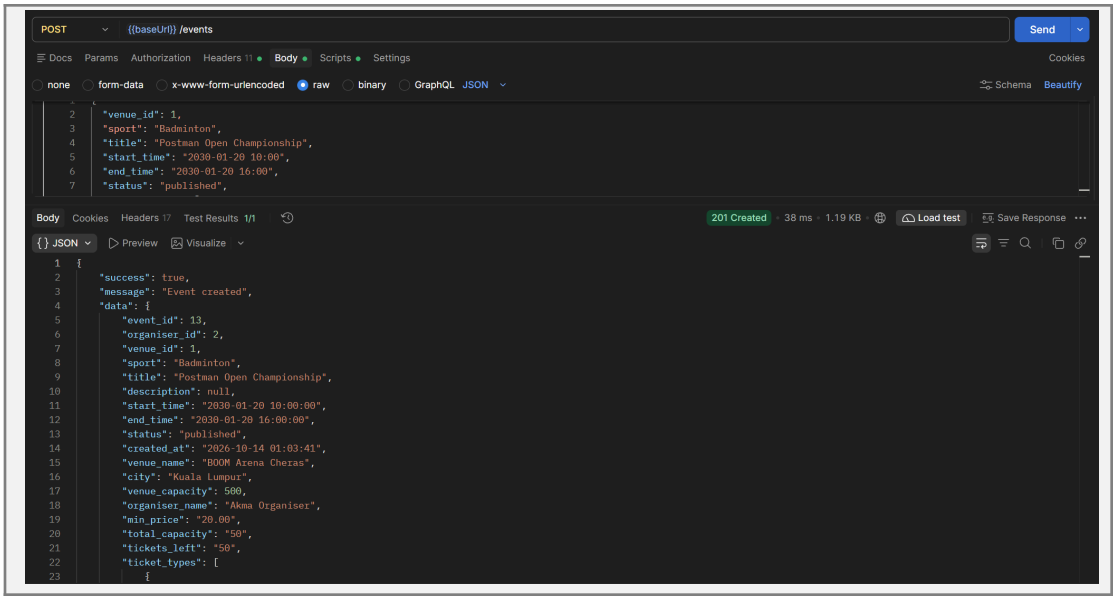


Figure 4.3: POST /events (201 Created)

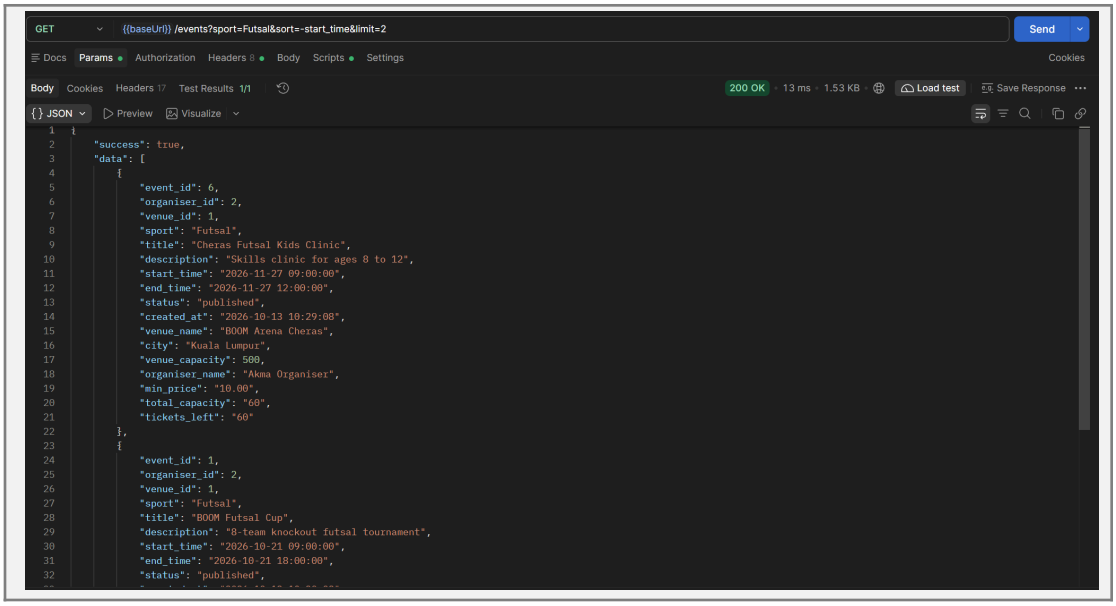


Figure 4.4: GET /events with sort and limit (200 OK)

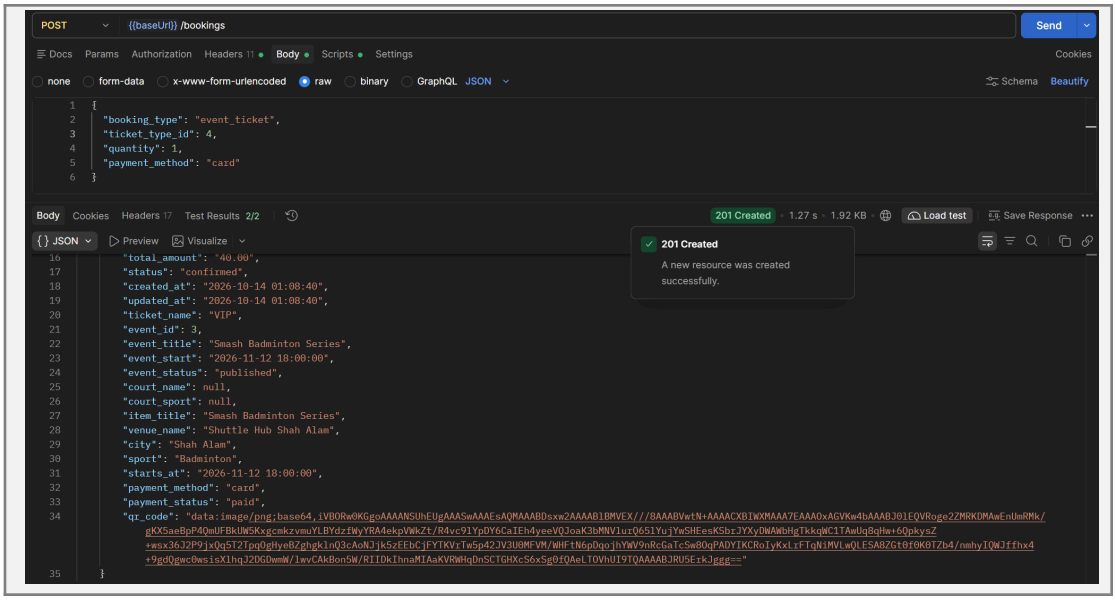


Figure 4.5: POST /bookings (201 Created with QR code)

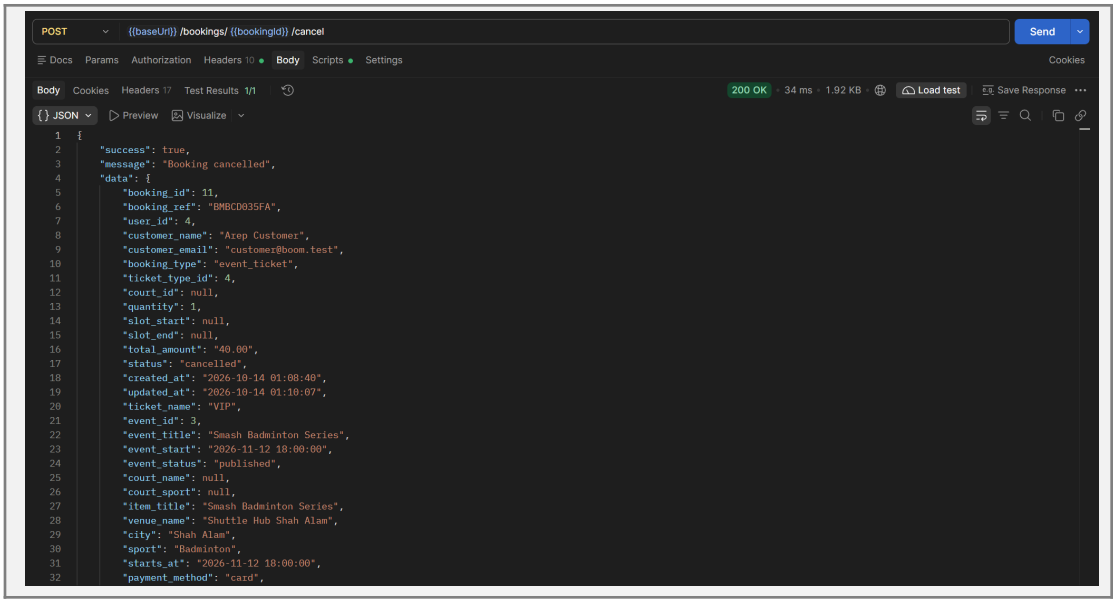


Figure 4.6: POST /bookings/{id}/cancel (200 OK)

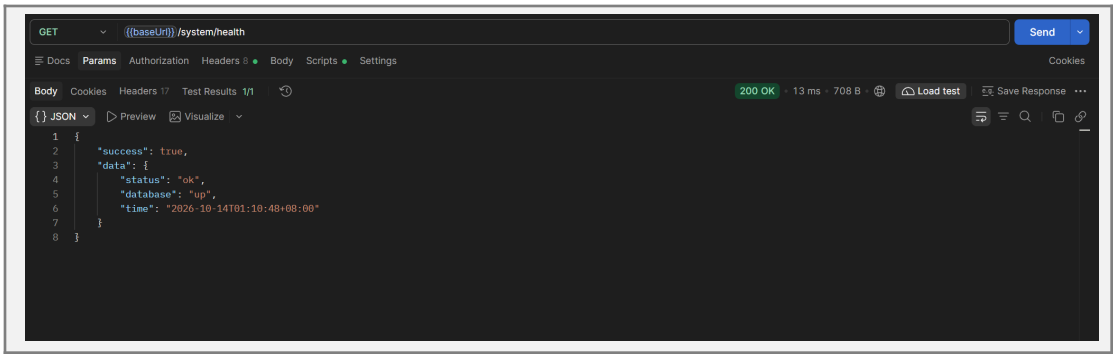


Figure 4.7: GET /system/health (200 OK)

4.4 ERROR RESPONSES

Request	Expected result	Figure
POST /auth/login with a wrong password	401 Invalid email or password	Figure 4.8
GET /users with a customer token	403 Forbidden	Figure 4.9
GET /bookings without a token	401 Missing token	Figure 4.10
POST /bookings with quantity 99	422 with errors.quantity	Figure 4.11
POST /bookings on a sold-out ticket	409 Sold out	Figure 4.12
POST /bookings for a court slot already taken	409 slot taken	Figure 4.13
POST /events overlapping another event at the same venue	409 schedule clash	Figure 4.14
GET /events/99999	404 Event not found	Figure 4.15
16 rapid POST /auth/login	429 with Retry-After	Figure 4.16

Table 4.3: Postman Test Suite – Error and Validation Test Cases

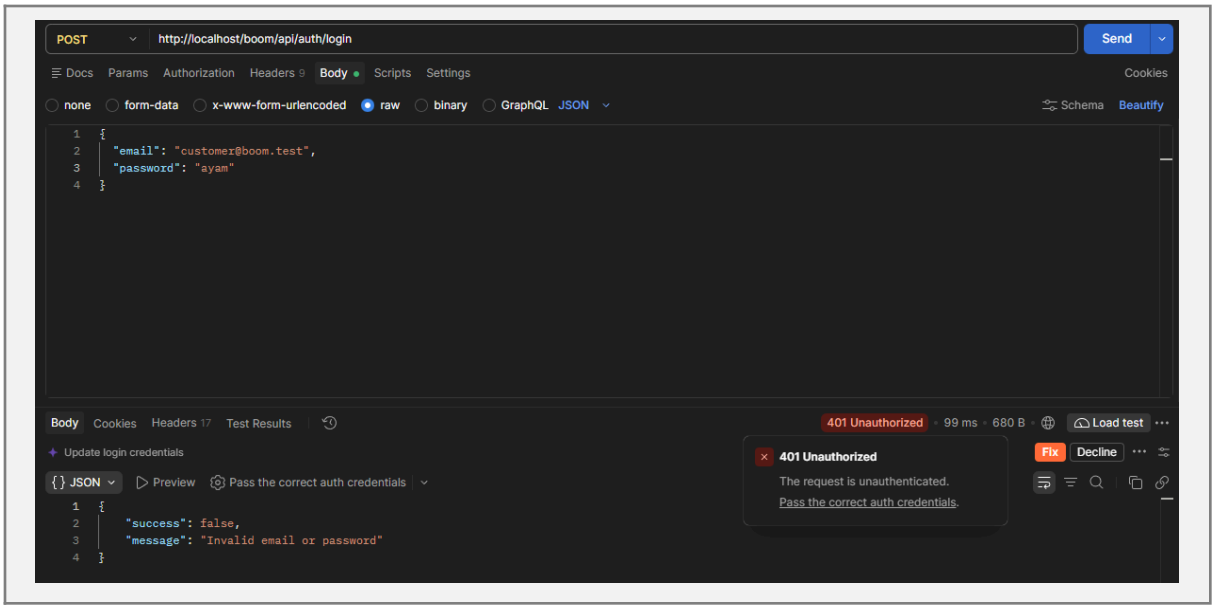


Figure 4.8: POST /auth/login wrong password (401)

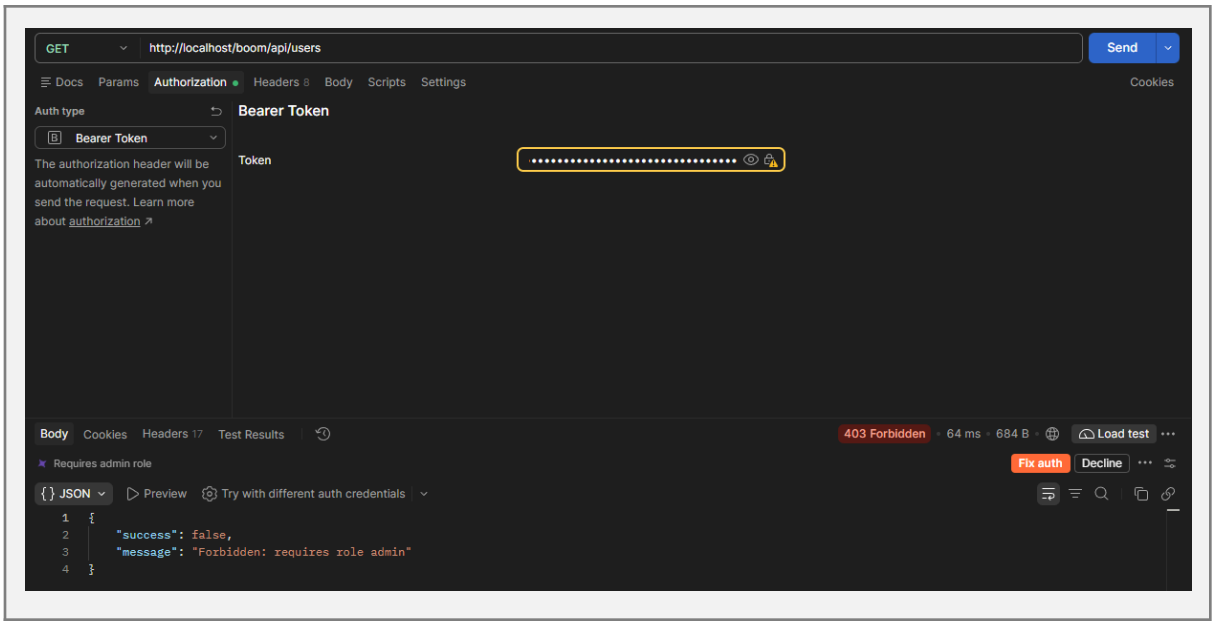


Figure 4.9: GET /users as customer (403)

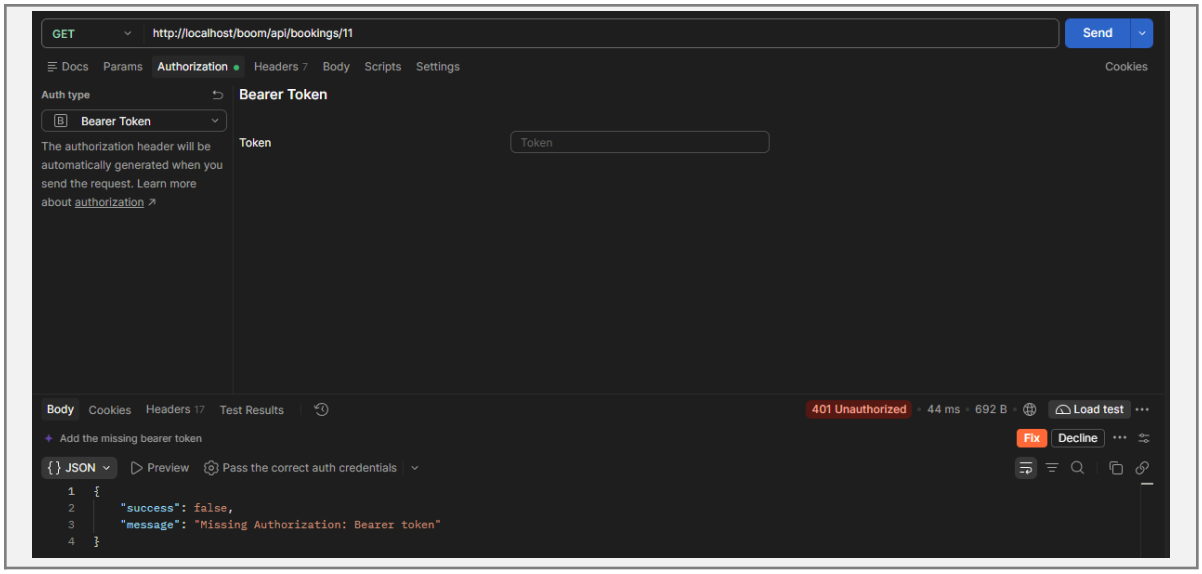


Figure 4.10: GET /bookings without token (401)

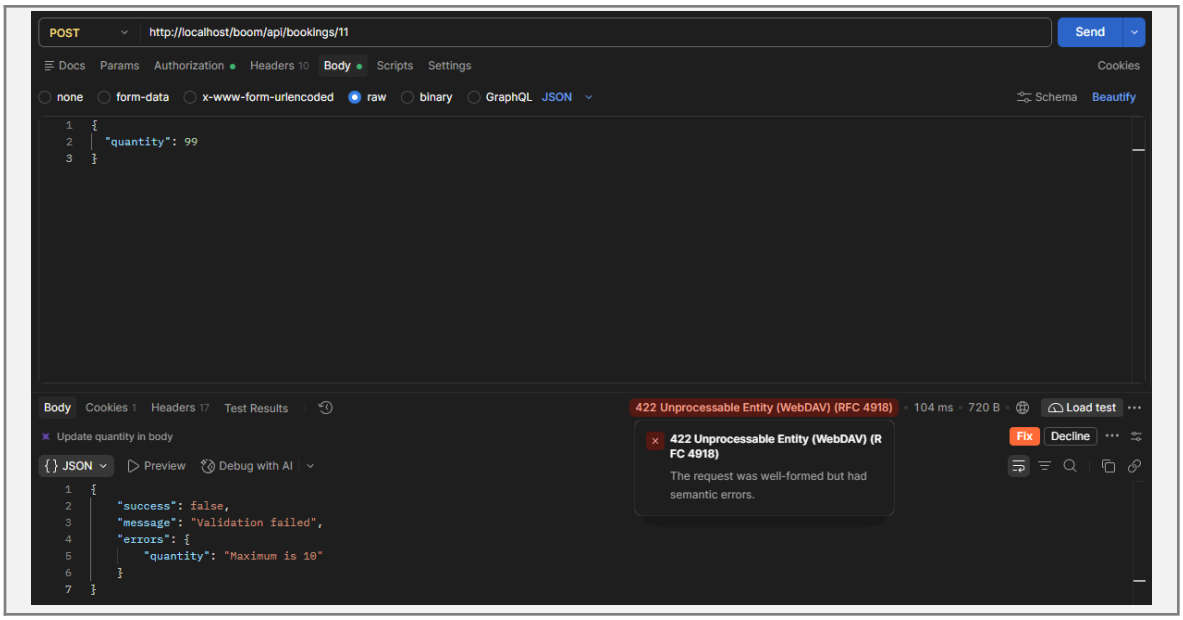


Figure 4.11: POST /bookings validation error (422)

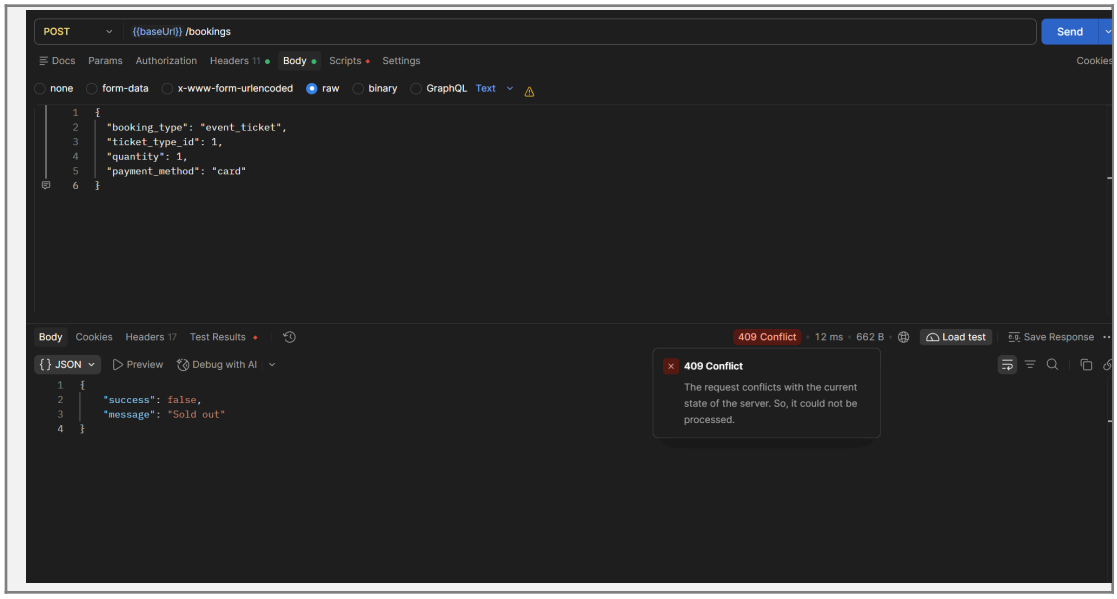


Figure 4.12: POST /bookings sold out (409)

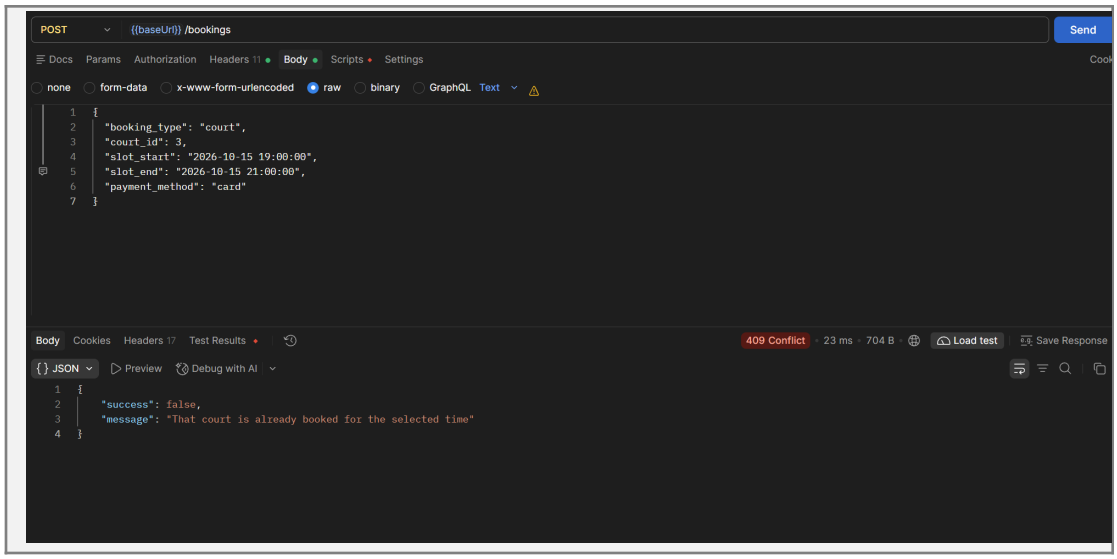


Figure 4.13: POST /bookings court slot taken (409)

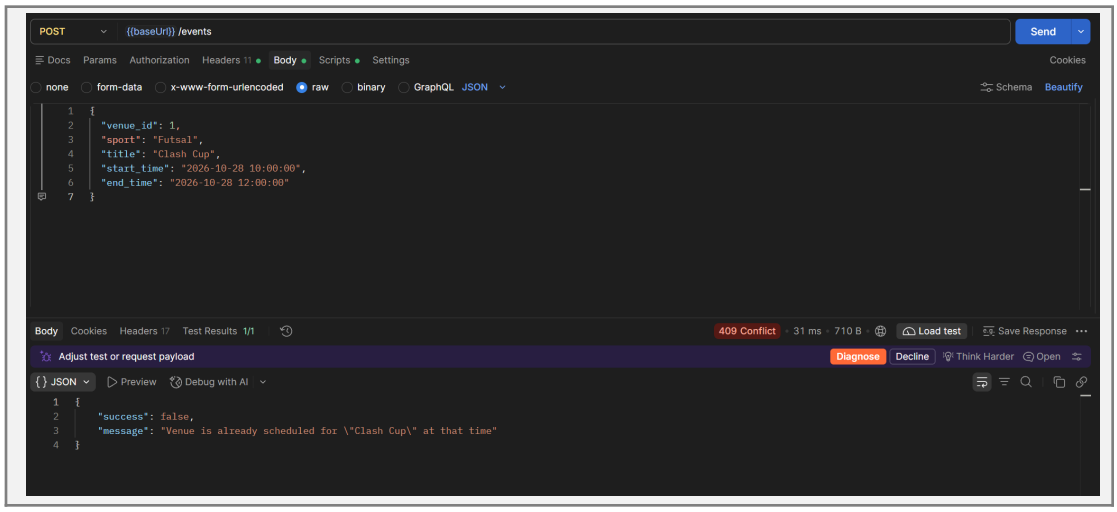


Figure 4.14: POST /events overlapping event (409)

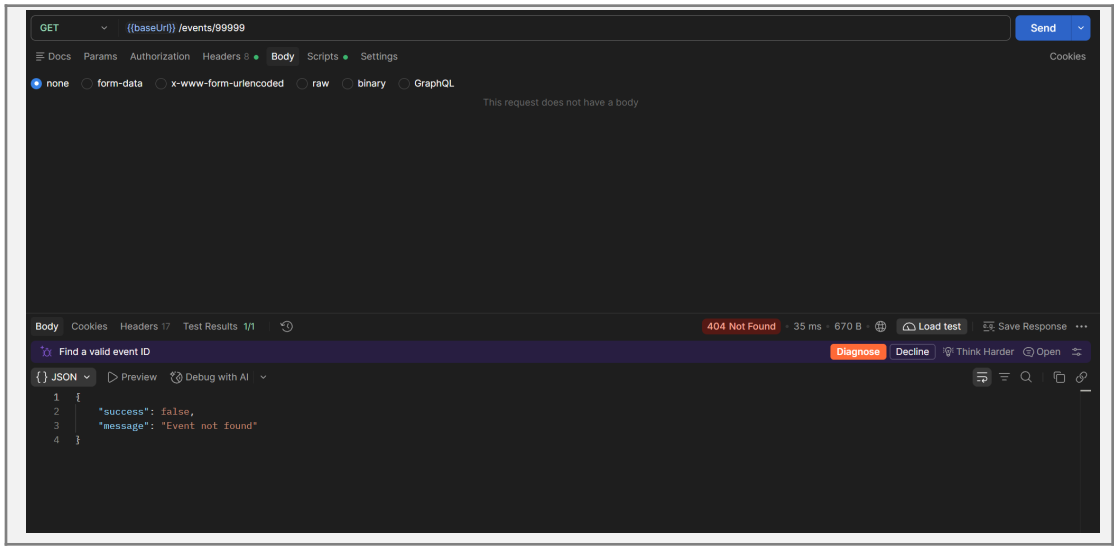


Figure 4.15: GET /events/99999 (404)

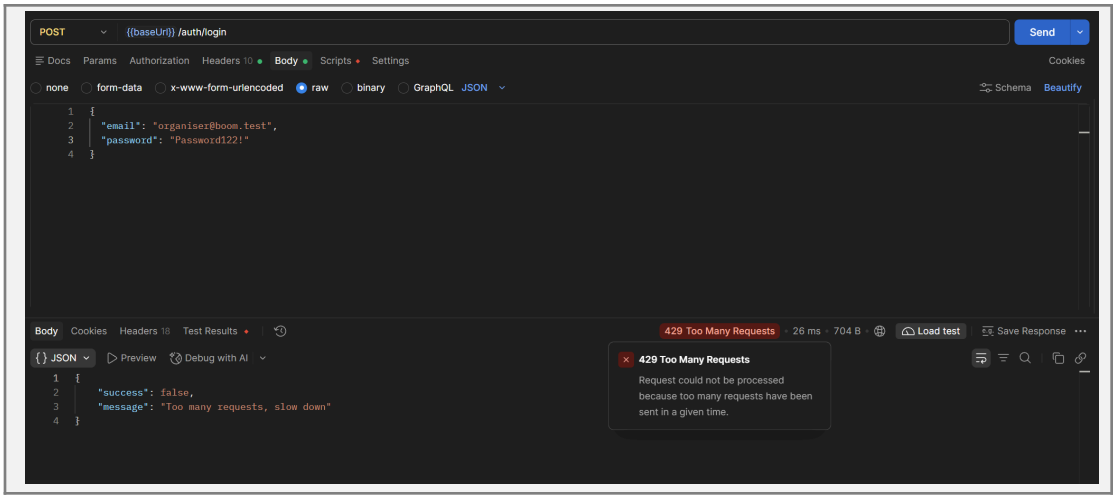


Figure 4.16: 16th login request (429 Too Many Requests)

5. SYSTEM IMPLEMENTATION

5.1 SHARED BACKEND IMPLEMENTATION

The backend is a single PHP REST API that all four frontends use. Apache mod_rewrite sends every request under /boom/api to api/index.php, which applies the middleware, finds the resource (auth, users, venues, events, bookings, system) and calls handle_<resource>() in api/routes/<resource>.php.

Folder / file	Responsibility
api/index.php, api/.htaccess	Front controller, URL routing, rate limiting, 404 handling, Authorization header fix for Apache
api/routes/	auth.php, users.php, venues.php, events.php, bookings.php, system.php: one file per resource
api/middleware/	security.php (headers, CORS), rate_limit.php, logger.php, error.php, validation.php, auth.php (JWT, RBAC, audit)
api/lib/	response.php (JSON envelope), query.php (filter, sort, pagination), jwt.php, qr.php, booking_service.php (cancel and refund)
config/config.php	Database, JWT, rate limit and QR settings, PDO connection
database/boom_plus.sql	DDL and DML scripts

Table 5.1: Backend Directory Structure and Component Responsibilities

Request pipeline: security headers → rate limit → request log → error handler → route → validation → JWT / RBAC check → business logic (PDO prepared statements) → JSON response.

Front controller (api/index.php):

```php
<?php

function request_segments(): array
{
    $path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH)
    $path = preg_replace('#^.*?/api/?#', '', $path)

    return array_values(array_filter(explode('/', trim($path, '/')), 'strlen'))
}


$segments = request_segments()
$method   = $_SERVER['REQUEST_METHOD']
$resource = $segments[0] ?? ''

apply_rate_limits($resource, $segments[1] ?? '')

if ($resource === '') {
    json_out(200, [
        'success'   => true,
        'name'      => 'BOOM+ API',
        'version'   => '2.0',
        'resources' => RESOURCES,
    ])
}

if (!in_array($resource, RESOURCES, true)) {
    json_error(404, 'Resource not found')
}

require_once __DIR__ . '/routes/' . $resource . '.php'

('handle_' . $resource)($method, array_slice($segments, 1))

json_error(404, 'Endpoint not found')
```

Authentication and RBAC (api/middleware/auth.php): the Bearer token is verified, the user is re-read from the database, and require_role() and require_owner_or_admin() protect routes.

```php
<?php
```

```
function optional_auth(): ?array
{
    $token = bearer_token()

    if (!$token) {
        return null
    }

    $claims = jwt_decode($token)

    if (!$claims) {
        json_error(401, 'Invalid or expired token')
    }

    $statement = db()->prepare(
        'SELECT user_id, full_name, email, phone, is_active, role FROM users
WHERE user_id = ?'
    )
    $statement->execute([$claims['sub']])
    $user = $statement->fetch()

    if (!$user || !$user['is_active']) {
        json_error(401, 'Account not found or disabled')
    }

    current_user_id((int) $user['user_id'])

    return $user
}

function require_auth(): array
{
    if (!bearer_token()) {
        json_error(401, 'Missing Authorization: Bearer token')
    }

    return optional_auth()
}

function require_role(string ...$roles): array
{
    $user = require_auth()

    if (!in_array($user['role'], $roles, true)) {
```

```
        json_error(403, 'Forbidden: requires role ' . implode(' or ', $roles))
    }

    return $user
}

function require_owner_or_admin(array $user, int $ownerId): void
{
    if ($user['role'] !== 'admin' && (int) $user['user_id'] !== $ownerId) {
        json_error(403, 'Forbidden: you do not own this resource')
    }
}
```

JWT (api/lib/jwt.php): tokens are signed with HMAC SHA-256, the algorithm is pinned to HS256 and the signature is compared in constant time.

PHP

```
<?php

function jwt_encode(array $claims): string
{
    $now = time()

    $header  = base64url_encode(json_encode(['alg' => 'HS256', 'typ' => 'JWT']))
    $payload = base64url_encode(json_encode($claims + ['iat' => $now, 'exp' =>
$now + JWT_TTL]))

    $unsigned = $header . '.' . $payload

    return $unsigned . '.' . jwt_sign($unsigned)
}

function jwt_decode(string $token): ?array
{
    $parts = explode('.', $token)

    if (count($parts) !== 3) {
        return null
    }

    [$header, $payload, $signature] = $parts

    $headerData = json_decode(base64url_decode($header), true)
```

```
    if (!is_array($headerData) || ($headerData['alg'] ?? '') !== 'HS256') {
        return null
    }

    if (!hash_equals(jwt_sign($header . '.' . $payload), $signature)) {
        return null
    }

    $claims = json_decode(base64url_decode($payload), true)

    if (!is_array($claims) || ($claims['exp'] ?? 0) < time()) {
        return null
    }

    return $claims
}
```

Booking transaction (api/routes/bookings.php): the ticket row is locked with FOR UPDATE inside a transaction so concurrent customers cannot oversell.

PHP

```php
<?php

function bookings_create(array $input): void
{
    $user = require_role('customer')

    $data = validate($input, [
        'booking_type'   => 'required|in:event_ticket,court',
        'payment_method' => 'required|in:card,fpx,ewallet,cash',
        'ticket_type_id' => 'int|min:1',
        'quantity'       => 'int|min:1|max:' . MAX_TICKETS_PER_BOOKING,
        'court_id'       => 'int|min:1',
        'slot_start'     => 'datetime',
        'slot_end'       => 'datetime',
    ])

    $pdo = db()
    $pdo->beginTransaction()

    $fields = $data['booking_type'] === 'event_ticket'
        ? prepare_ticket_booking($data)
        : prepare_court_booking($data)
```

```
    $bookingId = insert_booking((int) $user['user_id'], $fields,
$data['payment_method'])

    $pdo->commit()

    audit((int) $user['user_id'], 'BOOKING_CREATE', "booking $bookingId")

    $booking = find_booking($user, $bookingId)
    $booking['qr_code'] = booking_qr($booking)

    json_ok($booking, 'Booking created', 201)
}

function prepare_ticket_booking(array $data): array
{
    if (empty($data['ticket_type_id'])) {
        json_error(422, 'Validation failed', ['ticket_type_id' => 'Required'])
    }

    $quantity = $data['quantity'] ?? 1

    $statement = db()->prepare(
        'SELECT t.ticket_type_id, t.price, t.capacity, t.sold, e.status,
e.start_time
         FROM ticket_types t
         JOIN events e ON e.event_id = t.event_id
         WHERE t.ticket_type_id = ?
         FOR UPDATE'
    )
    $statement->execute([$data['ticket_type_id']])
    $ticket = $statement->fetch()

    if (!$ticket) {
        json_error(404, 'Ticket type not found')
    }

    assert_event_open($ticket)
    assert_seats($ticket, $quantity)

    db()->prepare('UPDATE ticket_types SET sold = sold + ? WHERE ticket_type_id =
?')
        ->execute([$quantity, $ticket['ticket_type_id']])

    return [
        'booking_type'   => 'event_ticket',
        'ticket_type_id' => $ticket['ticket_type_id'],
        'quantity'       => $quantity,
        'total_amount'   => round($ticket['price'] * $quantity, 2),
    ]
}
```

```
function assert_seats(array $ticket, int $quantity, int $alreadyHeld = 0): void
{
    $remaining = $ticket['capacity'] - $ticket['sold'] + $alreadyHeld

    if ($quantity > $remaining) {
        json_error(409, $remaining > 0 ? "Only $remaining ticket(s) left" : 'Sold
out')
    }
}

function insert_booking(int $userId, array $fields, string $paymentMethod): int
{
    [$bookingStatus, $paymentStatus, $paidAt] =
booking_status_for_payment($paymentMethod)

    $pdo = db()

    $pdo->prepare(
        'INSERT INTO bookings (booking_ref, user_id, booking_type,
ticket_type_id, court_id, quantity, slot_start, slot_end, total_amount, status)
         VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)'
    )->execute([
        new_booking_ref(),
        $userId,
        $fields['booking_type'],
        $fields['ticket_type_id'] ?? null,
        $fields['court_id'] ?? null,
        $fields['quantity'] ?? 1,
        $fields['slot_start'] ?? null,
        $fields['slot_end'] ?? null,
        $fields['total_amount'],
        $bookingStatus,
    ])

    $bookingId = (int) $pdo->lastInsertId()

    $pdo->prepare('INSERT INTO payments (booking_id, amount, method, status,
paid_at) VALUES (?, ?, ?, ?, ?)')
        ->execute([$bookingId, $fields['total_amount'], $paymentMethod,
$paymentStatus, $paidAt])

    return $bookingId
}
```

Cancel and refund (api/lib/booking_service.php):

```
PHP
<?php

function release_booking(array $booking, string $newStatus = 'cancelled'): void
```

```
{
    $pdo = db()

    $pdo->prepare('UPDATE bookings SET status = ? WHERE booking_id = ?')
        ->execute([$newStatus, $booking['booking_id']])

    if ($booking['booking_type'] === 'event_ticket' &&
$booking['ticket_type_id']) {
        $pdo->prepare('UPDATE ticket_types SET sold = GREATEST(0, sold - ?) WHERE
ticket_type_id = ?')
            ->execute([$booking['quantity'], $booking['ticket_type_id']])
    }

    $pdo->prepare("UPDATE payments SET status = IF(status = 'paid', 'refunded',
'failed') WHERE booking_id = ? AND status IN ('paid','pending')")
        ->execute([$booking['booking_id']])
}
```

5.2 HOW THE FRONTENDS COMMUNICATE WITH THE API

Every frontend is a server-side PHP portal. After a successful POST /auth/login, login_user() stores the JWT and the user in the PHP session (session_regenerate_id prevents session fixation). Each page then calls the shared helper api() in shared/bootstrap.php, which sends an HTTP request to API_BASE with Authorization: Bearer <token> and decodes the JSON. If the API answers 401 the session is destroyed and the user returns to the login page. Forms contain a CSRF token and all output is escaped with e().

PHP

```
<?php

function api(string $method, string $path, ?array $data = null, bool
$authenticated = true): array
{
    $headers = "Content-Type: application/json\r\nAccept: application/json\r\n"
        . 'X-Forwarded-For: ' . ($_SERVER['REMOTE_ADDR'] ?? '127.0.0.1') . "\r\n"

    if ($authenticated && !empty($_SESSION['token'])) {
        $headers .= "Authorization: Bearer {$_SESSION['token']}\r\n"
    }

    $context = stream_context_create([
        'http' => [
            'method'        => $method,
```

```
            'header'        => $headers,
            'content'       => $data !== null ? json_encode($data) : '',
            'ignore_errors' => true,
            'timeout'       => 20,
        ],
    ])

    $raw      = @file_get_contents(API_BASE . $path, false, $context)
    $response = json_decode($raw ?: '', true) ?: ['success' => false, 'message'
=> 'API unreachable. Start Apache and MySQL in XAMPP.']

    preg_match('/\s(\d{3})\s/', $http_response_header[0] ?? '', $matches)
    $response['_status'] = (int) ($matches[1] ?? 0)

    if ($response['_status'] === 401 && $authenticated &&
!empty($_SESSION['token'])) {
        session_destroy()
        redirect('login.php?expired=1')
    }

    return $response
}

function api_get(string $path, array $query = [], bool $authenticated = true):
array
{
    $query = array_filter($query, fn ($value) => $value !== '' && $value !==
null)

    return api('GET', $path . ($query ? '?' . http_build_query($query) : ''),
null, $authenticated)
}

function login_user(array $data): void
{
    session_regenerate_id(true)

    $_SESSION['token'] = $data['token']
    $_SESSION['user']  = $data['user']
}

function csrf_check(): void
{
    if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ?? '')) {
        http_response_code(400)
        exit('Bad CSRF token')
    }
}
```

Step	What happens
1	The customer submits the booking form (POST, with CSRF token) to a portal page such as Arif/event.php.
2	The page validates the CSRF token and calls api('POST', '/bookings', [...]) with the session JWT.
3	The API middleware checks the rate limit, logs the request and verifies the token and the customer role.
4	The route validates the body, locks the ticket row, inserts the booking and payment, and returns JSON (201).
5	The portal reads success, errors and data: on success it redirects to the ticket page on 409 or 422 it shows the message beside the form.

Table 5.2: Frontend-to-API Communication Lifecycle Steps

The same four steps apply to every page. Because all portals use the same helper and the same API, a new frontend only needs to call the endpoints in Section 3.

5.3 FRONTEND OF HAFIQ: Authentication, User Profile and User Administration

Student: Hafiq (Database & Security Lead)

The Hafiq portal contains everything relating to identity, specifically: login and registration page, password page, profile page and two admin tools, user management and audit log. JWT session management, multiple strong password messages and CSRF protection are demonstrated.

Page / file	Purpose	API endpoints called
Hafiq/login.php, register.php, forgot.php, reset.php	Account access	POST /auth/login, /auth/register, /auth/forgot-password, /auth/reset-password
Hafiq/profile.php	View and update profile, change password	GET, PUT /users/me PUT /users/me/password
Hafiq/users.php (admin)	Search, create, change role, enable or disable, delete users	GET, POST /users PUT /users/{id}/role, /status DELETE /users/{id}
Hafiq/audit.php (admin)	Audit trail with filters	GET /system/audit-logs
Hafiq/dashboard.php, events.php, bookings.php	Role dashboard and lists	GET /system/stats, /events, /bookings

Table 5.3: Hafiq Frontend Portal Files and Mapped API Endpoints

The shared files at shared/pages/, login, register, forgot and reset password, profile and dashboard and viewing event and booking lists are all reusable by portals and they call to /auth/*, /users/me, /system/stats, /events and /bookings.

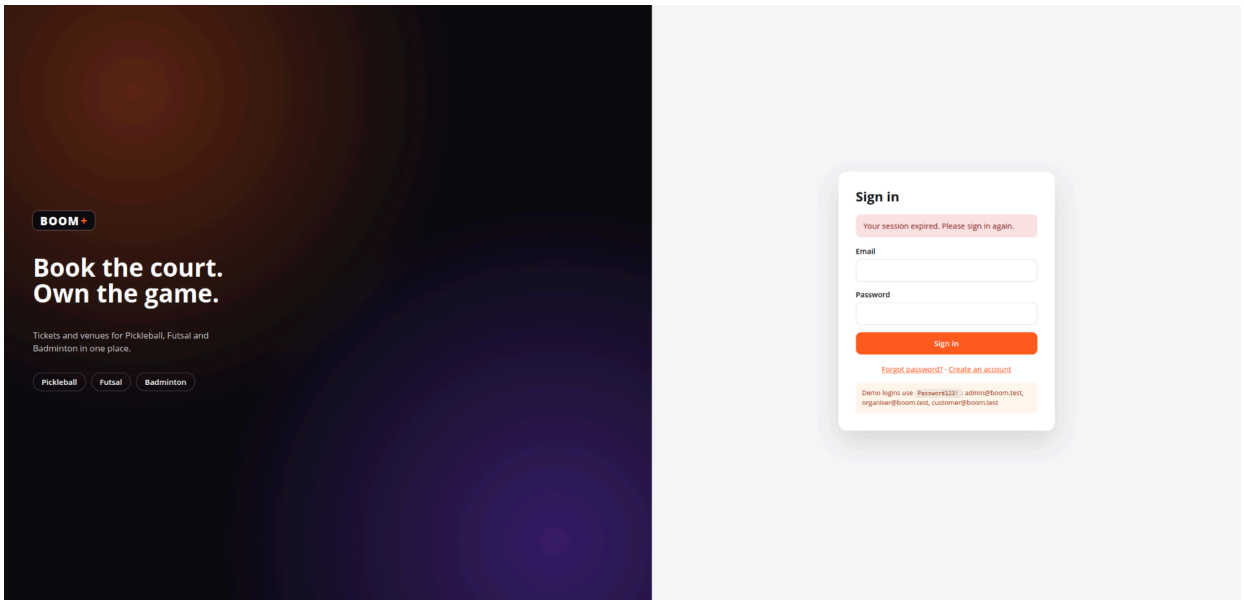


Figure 5.2: Hafiq: login page

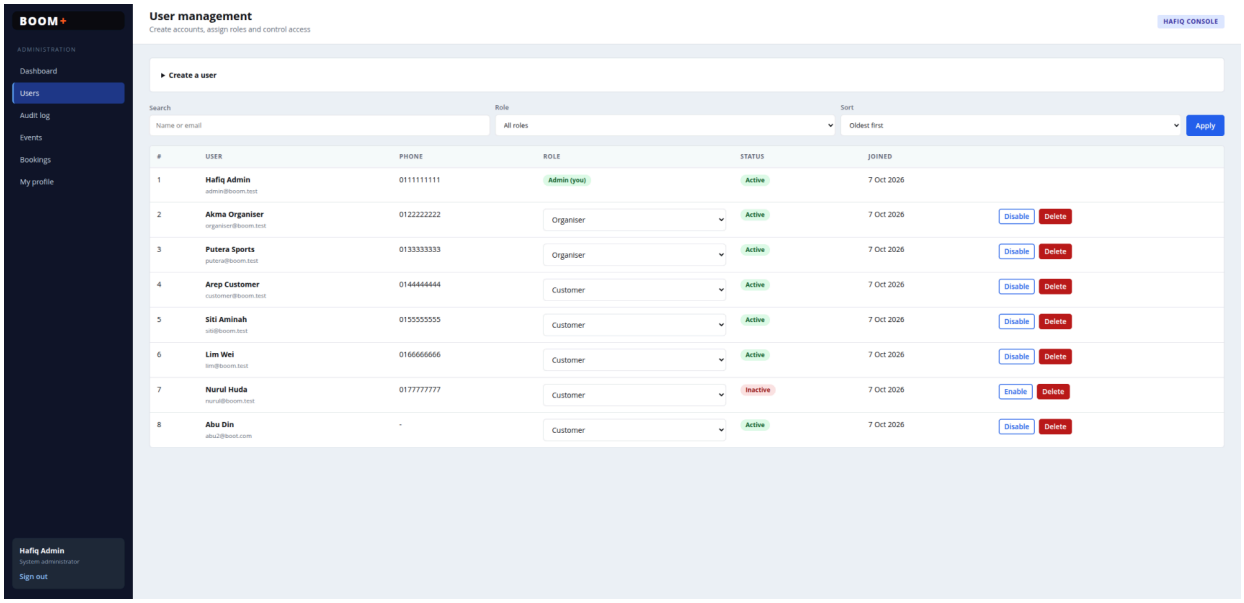


Figure 5.3: Hafiq: user management page (admin)

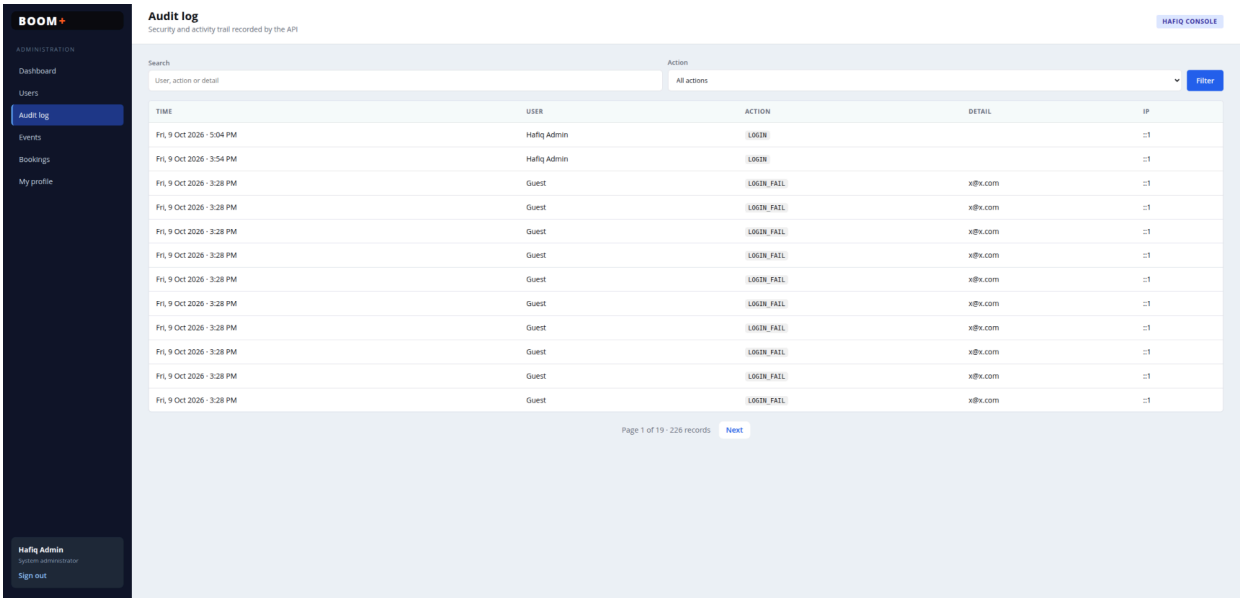


Figure 5.4: Hafiq: audit log page (admin)

5.4 FRONTEND OF AKMA: Organiser / Admin Management Dashboard

Student: Akma (API Documentation & Organiser Dashboard)

On the management side Akma portal allows organisers/administrators to manage/maintain venues, courts, events and ticket types. Forms display validation and conflict messages returned by the API (schedule clash, capacity above venue limit).

Page / file	Purpose	API endpoints called
Akma/venues.php, venue.php, venue_form.php	List, create, edit, activate and delete venues manage courts view daily schedule	GET, POST /venues PUT, PATCH, DELETE /venues/{id} POST, PUT, DELETE /venues/{id}/courts GET /venues/{id}/schedule
Akma/manage_events.php	List events, publish, cancel, delete	GET /events PATCH, DELETE /events/{id}
Akma/event_form.php	Create or edit an event with ticket types	POST, PUT /events POST, PUT, DELETE /events/{id}/tickets
Akma/bookings.php, dashboard.php	Bookings of the manager and statistics	GET /bookings, /system/stats

Table 5.4: Akma Organiser Dashboard Files and Mapped API Endpoints

In addition, each portal reuses the shared pages in shared/pages/(login, register, forgot and reset password, profile, dashboard, event and booking lists), which call /auth/*, /users/me, /system/stats, /events and /bookings.

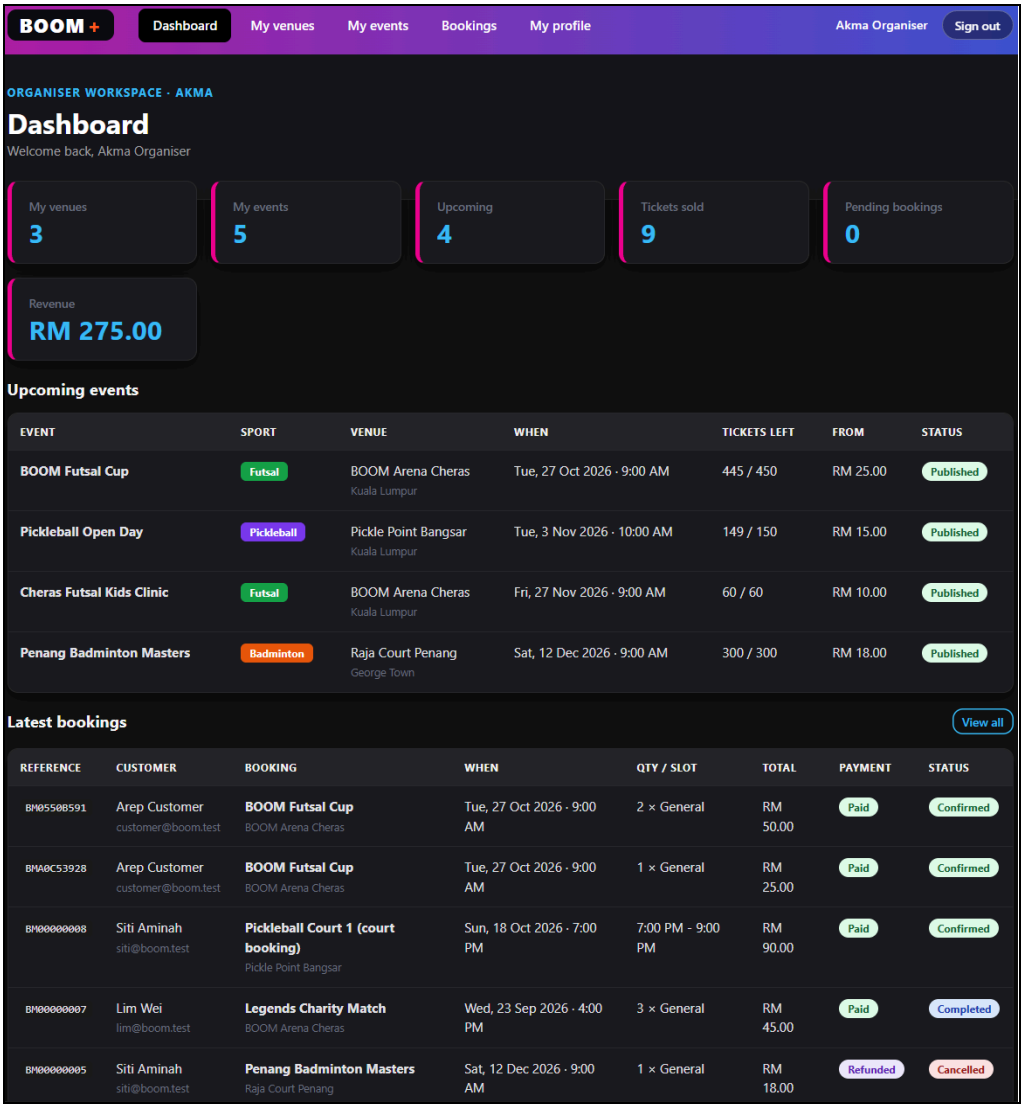


Figure 5.5: Organiser Workspace Dashboard

Figure 5.5 shows the main Organiser Workspace Dashboard for the authenticated organiser account. This interface fetches live aggregated metrics via the backend API, displaying key operational statistics such as registered venues, active events, ticket sales count, and total revenue. It also features quick-access tabular overviews for upcoming sports events and recent customer reservations, allowing organisers to monitor daily facility performance at a glance.

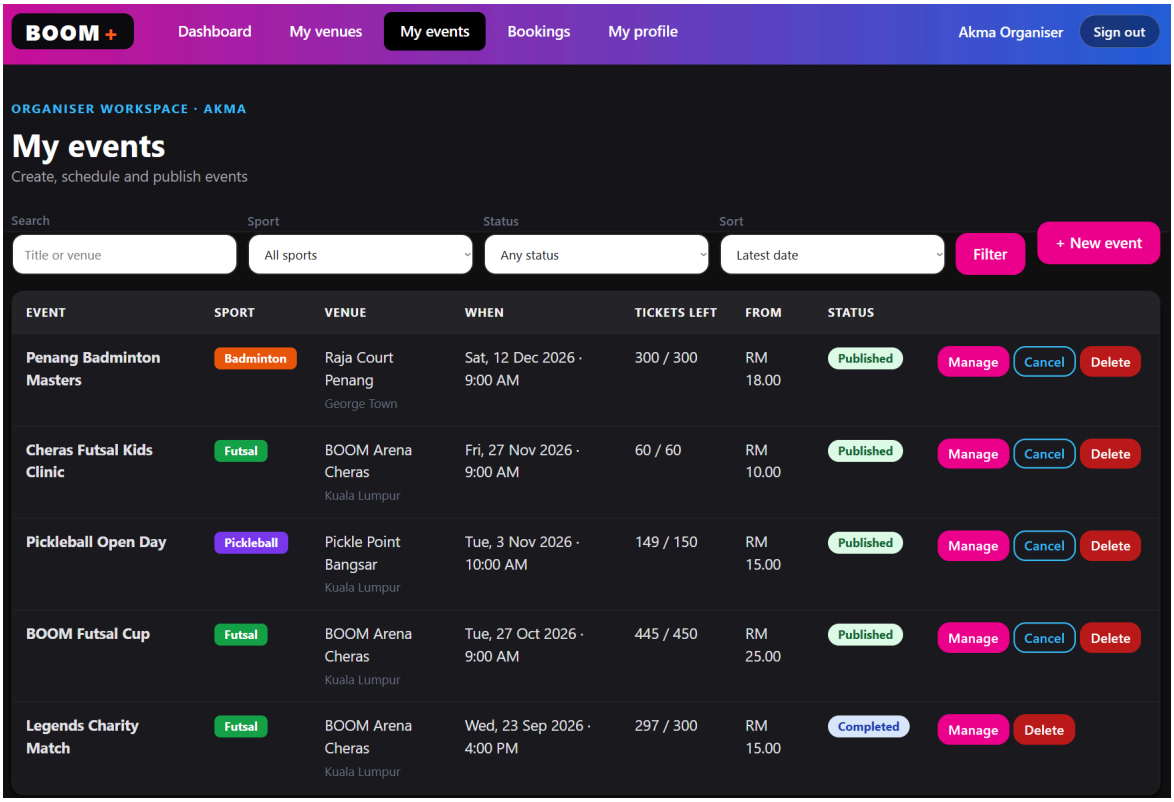


Figure 5.6: Event Scheduling & Management

Figure 5.6 displays the event and court scheduling interface powered by the `/events` API endpoints. Organisers can create bookable sessions, coaching clinics, or tournaments, complete with start schedules, ticket capacities, base pricing, and publishing states. The page enables full lifecycle management through filter toolbars and action controls, allowing organisers to update details, transition event statuses, or cancel events while maintaining data integrity.

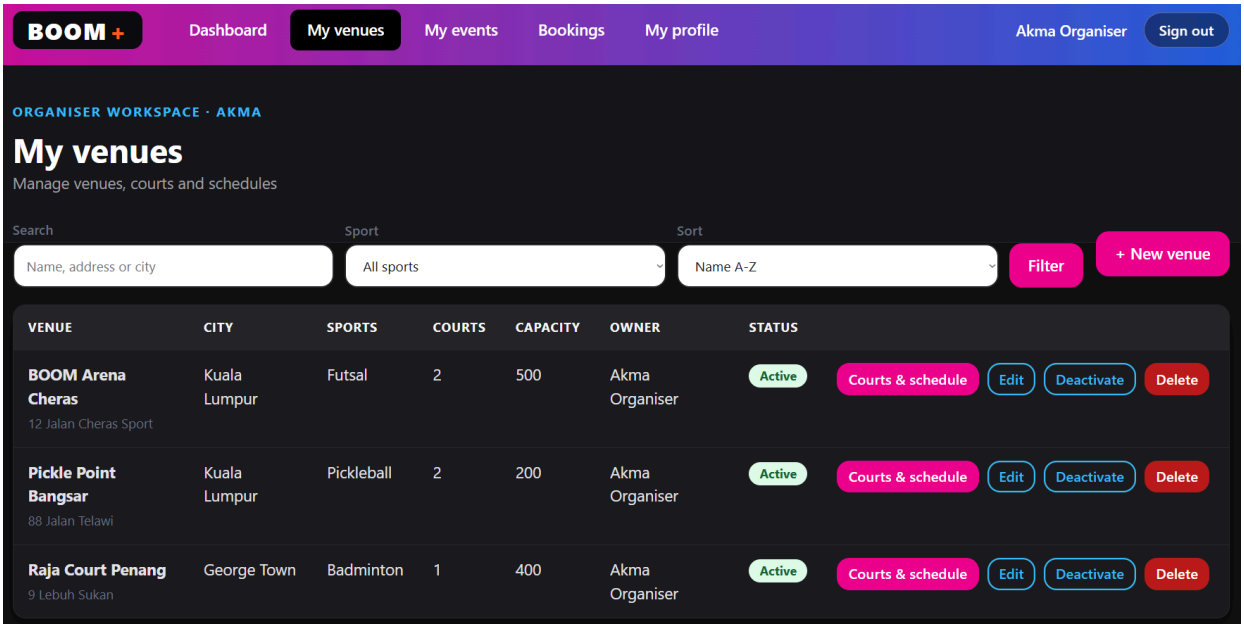


Figure 5.7: Venue Management

Figure 5.7 illustrates the venue management interface communicating with the `/venues` resource endpoints. Organisers can view all owned sports facilities alongside details such as court numbers, capacity limits, and operating status. The interface integrates API management middleware with real-time text searching, sport categorization (Futsal, Pickleball, Badminton), and sorting options, while providing full CRUD action controls to add, edit, schedule, or deactivate venues.

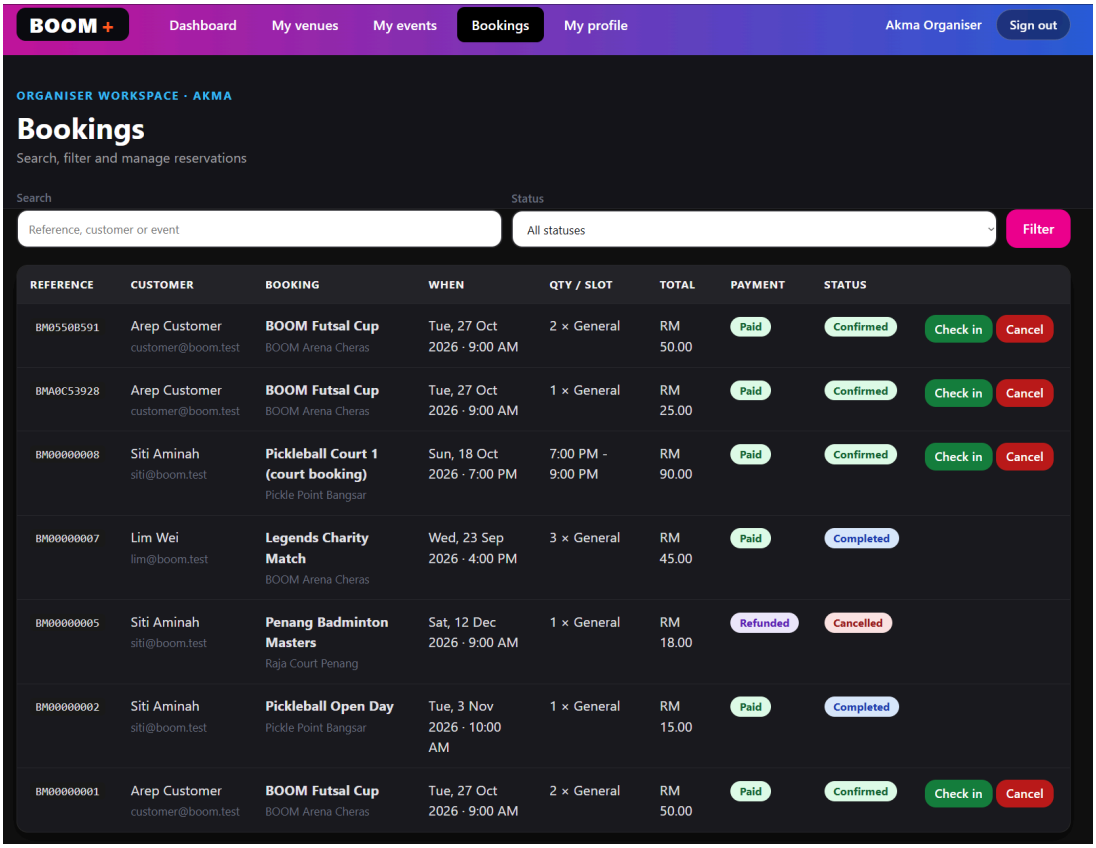


Figure 5.8: Reservations & Attendance Tracking

Figure 5.8 demonstrates the organiser reservations tracking portal that interfaces with the `/bookings` endpoints. It provides a real-time ledger of customer bookings tied to the organiser's events, displaying reference codes, slot times, payment states, and ticket quantities. Organisers can filter reservations, process attendee check-ins on match day, or handle order cancellations directly from the management table.

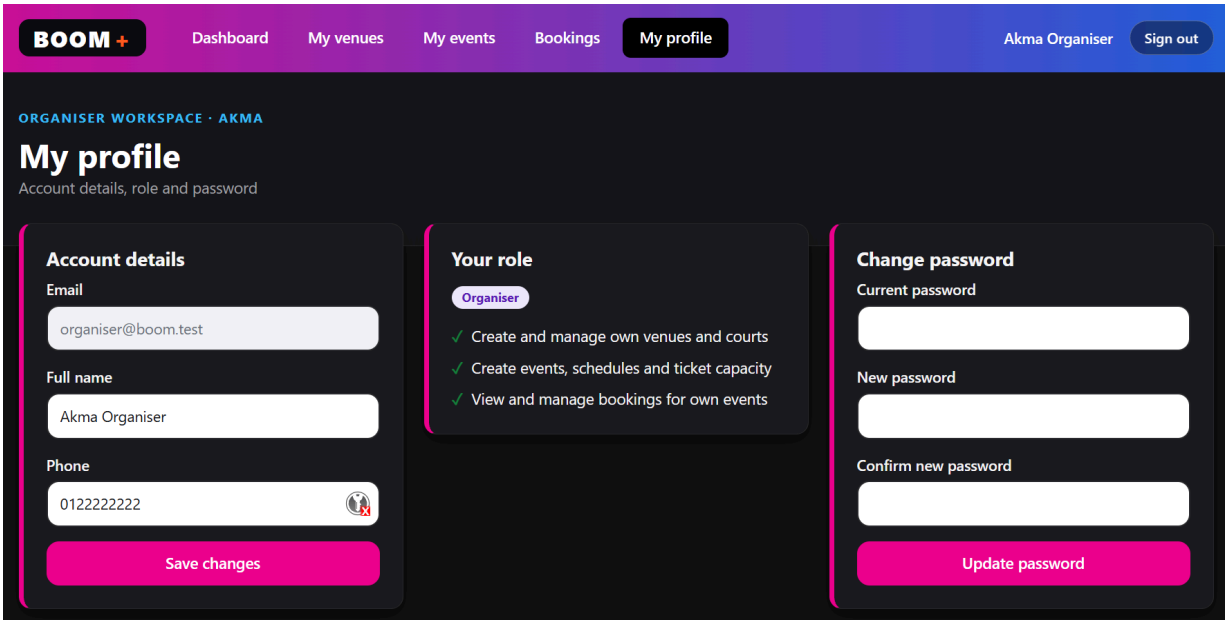


Figure 5.9: User Profile & Role Verification

Figure 6.5 depicts the account management and role-based access control (RBAC) overview screen. This interface displays verified profile credentials alongside granted organiser system permissions, validating that the active JWT authentication token restricts administrative venue and event management access to authorized roles. It also provides self-service forms for updating personal contact details and changing account passwords.

5.5 FRONTEND OF ARIF: Customer Booking & Checkout Portal

Student: Arif / Arep (Customer Booking, QR e-ticket and API Testing)

The customer booking journey is the Arif portal: they browse events, then see their live availability, choose their ticket type and payment method, then actually book a court from looking view their e-ticket including the QR code. Organisers check the check in page on this system to confirm a booking reference so they can check them in and out of courts and mark jobs as completed.

Page / file	Purpose	API endpoints called
Arif/events.php	Browse and filter events	GET /events, /venues
Arif/event.php	Event detail, availability and ticket booking	GET /events/{id}, /events/{id}/availability POST /bookings
Arif/courts.php	Choose venue, date and hours, book a court	GET /venues, /venues/{id}/schedule POST /bookings
Arif/tickets.php, booking.php	My tickets and e-ticket with QR code	GET /bookings?view=active, /bookings/{id}
Arif/verify.php (organiser)	Check-in by booking reference	GET /bookings?q=ref PATCH /bookings/{id}/status

Table 5.5: Arif Customer Booking Portal Files and Mapped API Endpoints

Each portal also reuses the shared pages in /shared/pages/(login, register, forgotandresetpassword, profile, dashboard, eventandbookinglists) which call /auth/*, /users/me, /system/stat/eventsand/bookings.

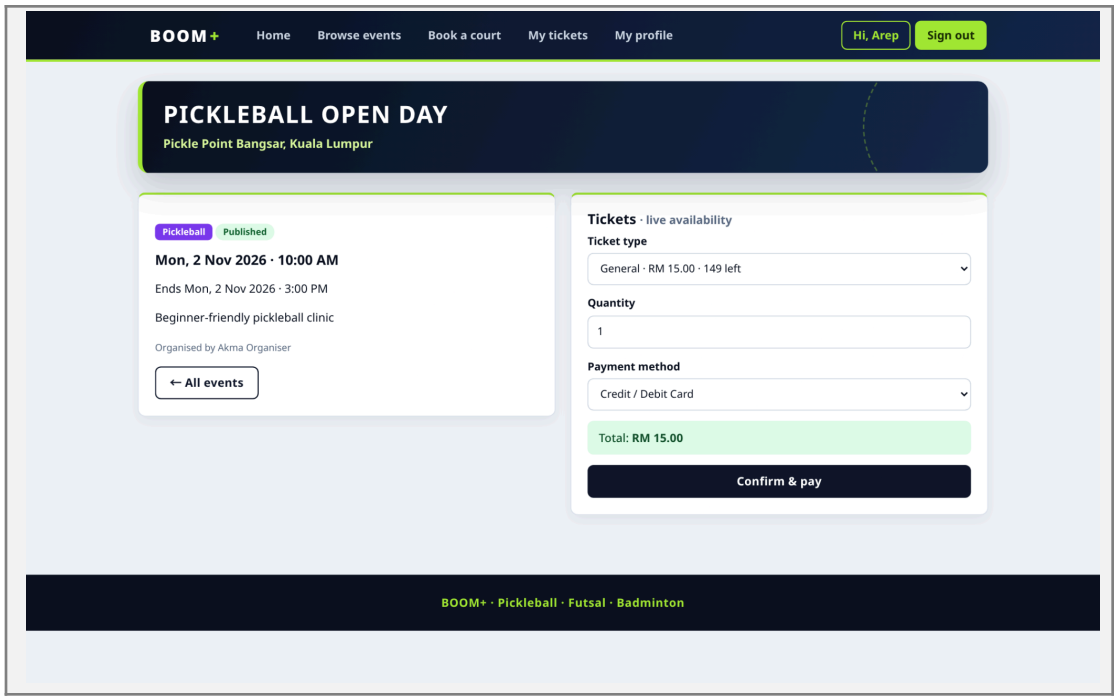


Figure 5.10: Arif: event booking page

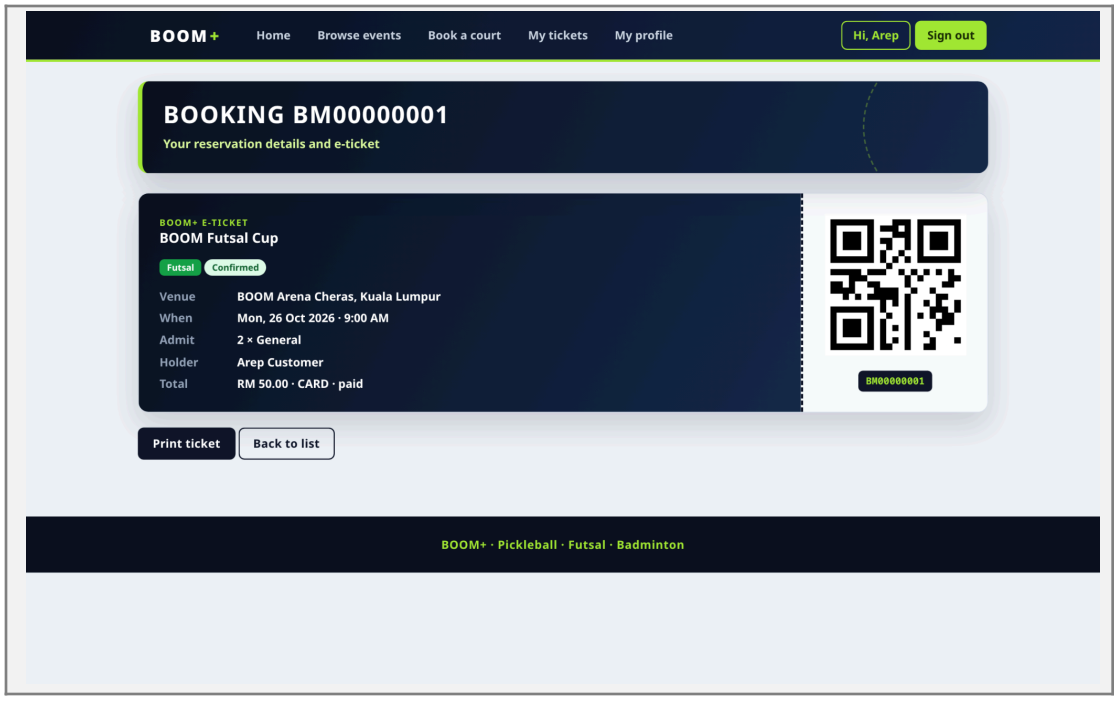


Figure 5.11: Arif: e-ticket with QR code

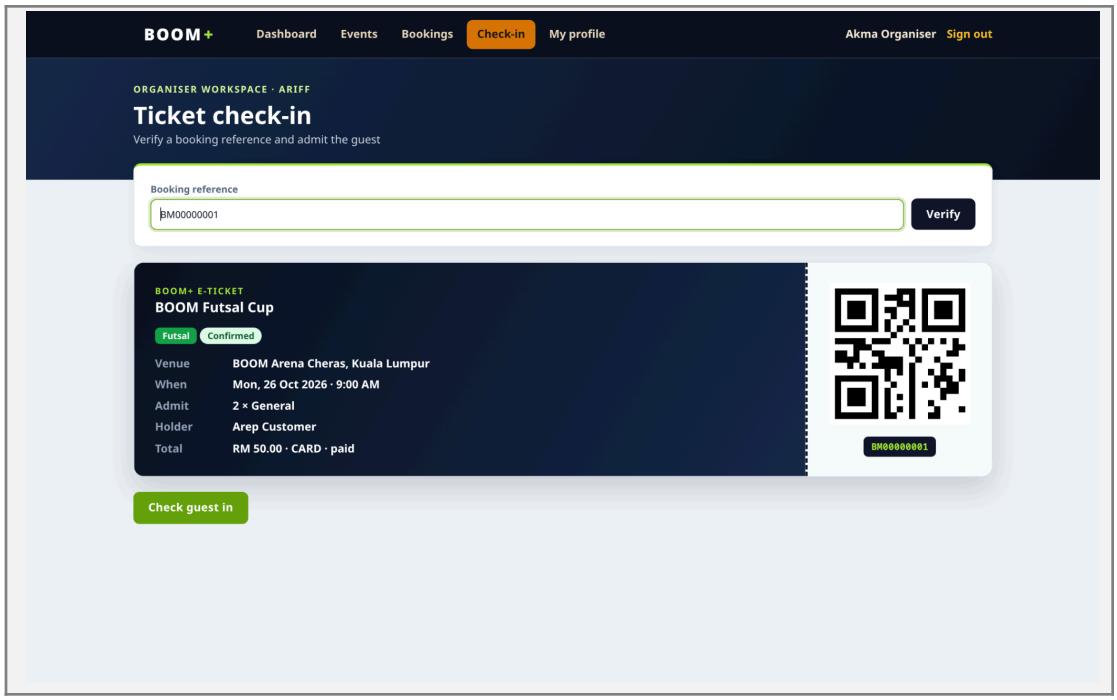


Figure 5.12: Arif: organiser check-in page

5.6 FRONTEND OF PUTERA AIDIL: Customer Account & Ticketing Portal

Student: Putera Aidil (Debugging, Performance and Booking History)

Putera_Aidil focuses on bookings made by customers, which show a customer's active list, history and modifying a booking (quantity of courts, court location or time). Addition for administrator is API request log, showing duration and status of each call, essential for performance work highlighted in section 9.

Page / file	Purpose	API endpoints called
Putera_Aidil/bookings.php	Active bookings	GET /bookings
Putera_Aidil/history.php	Past and cancelled bookings	GET /bookings?view=history
Putera_Aidil/booking.php	Booking detail, modify, cancel	GET /bookings/{id} PUT /bookings/{id} POST /bookings/{id}/cancel
Putera_Aidil/logs.php (admin)	API request log, slow requests and error rate	GET /system/request-logs

Table 5.6: Putera Aidil Account and Logging Portal Files and Mapped API Endpoints

All portals also reuse the shared pages in shared/pages/ (login, register, forgot and reset password, profile, dashboard, event and booking lists), which call /auth/*, /users/me, /system/stats, /events and /bookings.

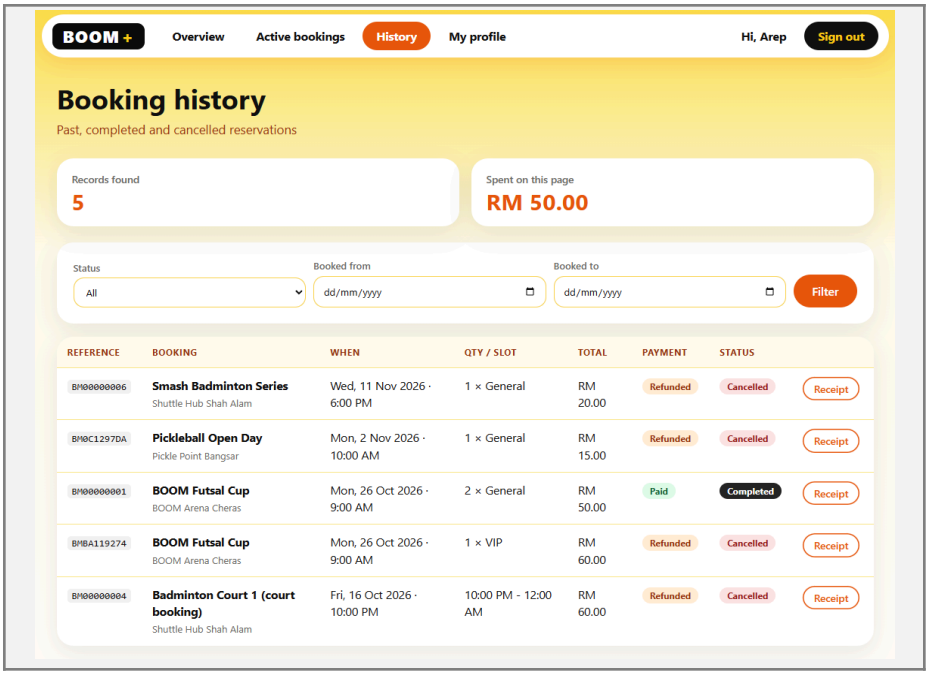


Figure 5.13: Putera Aidil: booking history page

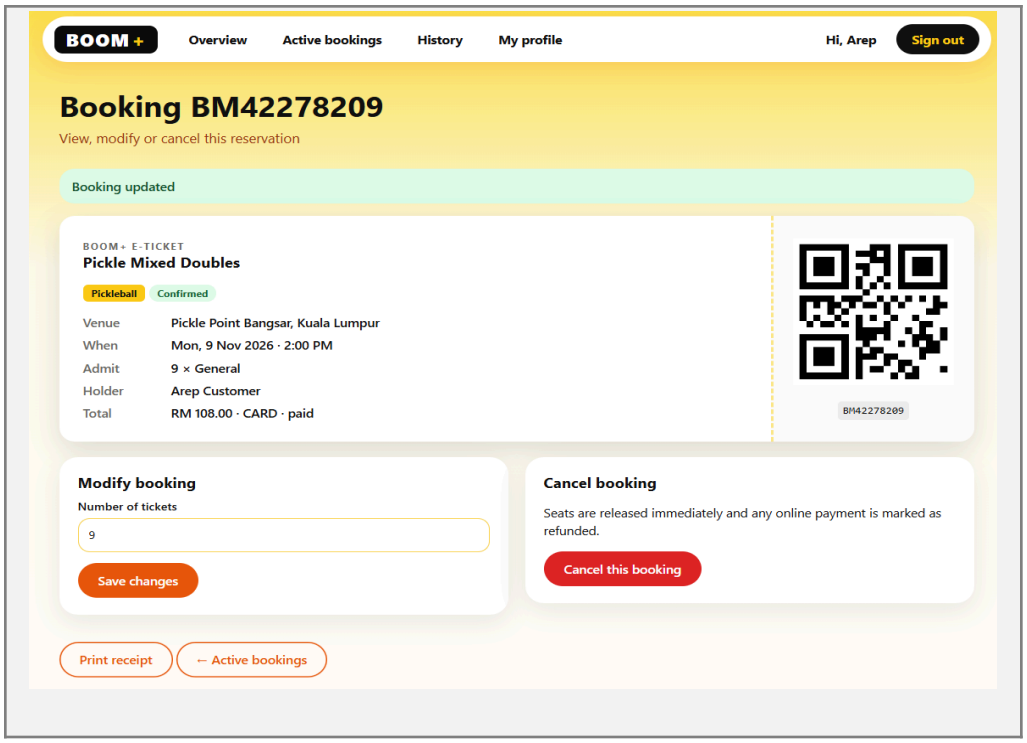


Figure 5.14: Putera Aidil: modify booking page

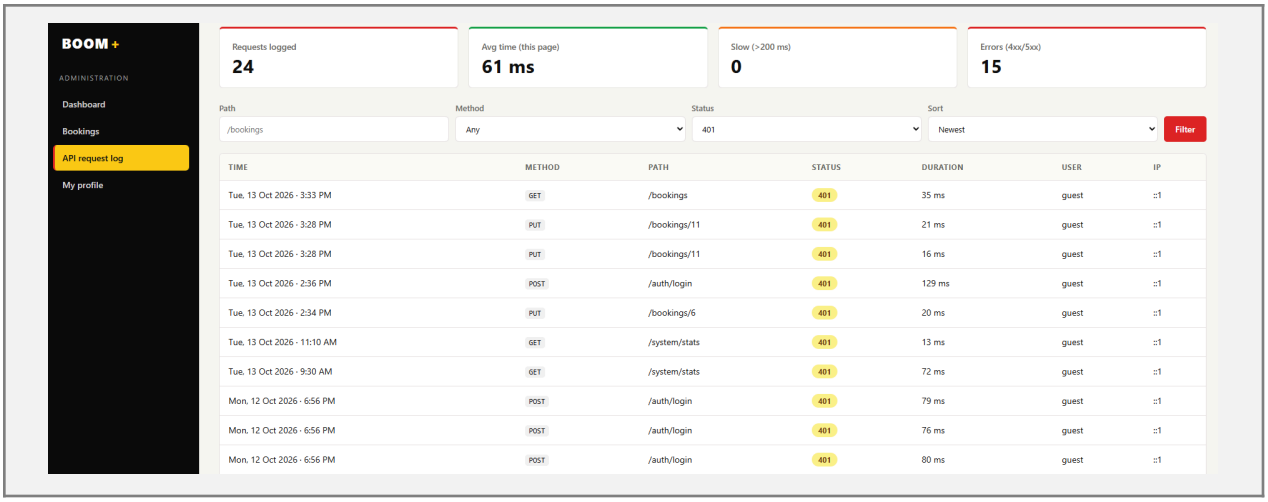


Figure 5.15: Putera Aidil: API request log (admin)

6. SYSTEM DEMONSTRATION

The screenshots below show the running system on XAMPP (http://localhost/boom/). Demo accounts: admin@boom.test, organiser@boom.test, customer@boom.test (password Password123!).

6.1 Login and role-based dashboards

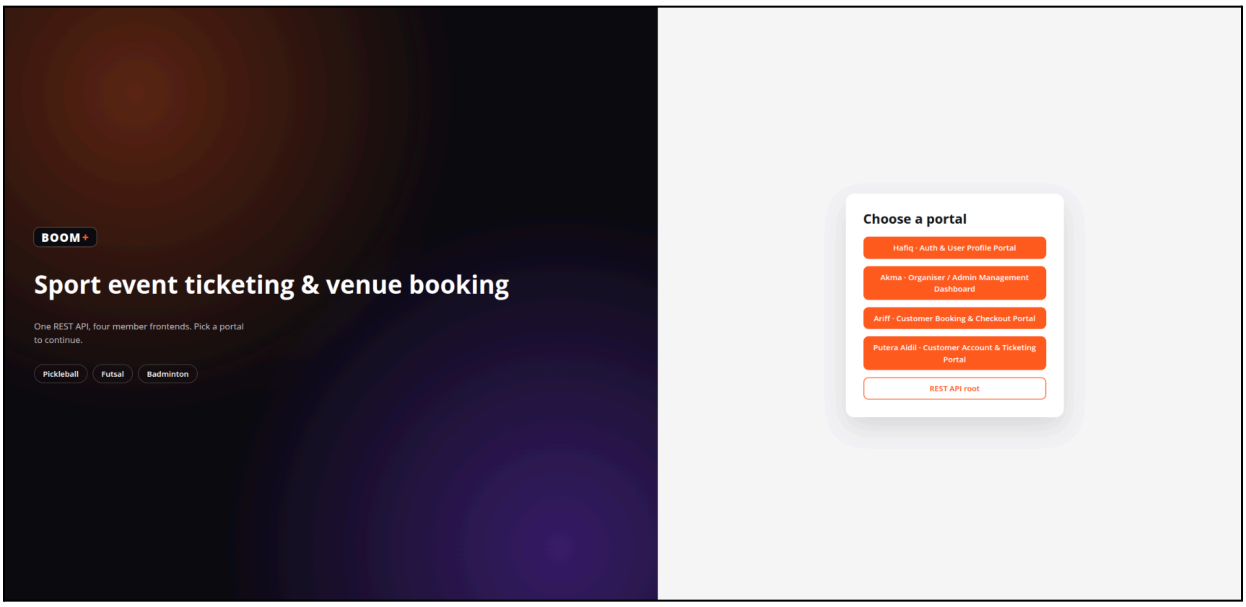


Figure 6.1: Login and role-based dashboards

The user submits the login form. The portal calls POST /auth/login, stores the JWT in the session and opens the dashboard for the role. The dashboard numbers come from GET /system/stats, which returns different figures for admin, organiser and customer.

6.2 Browsing events and live availability

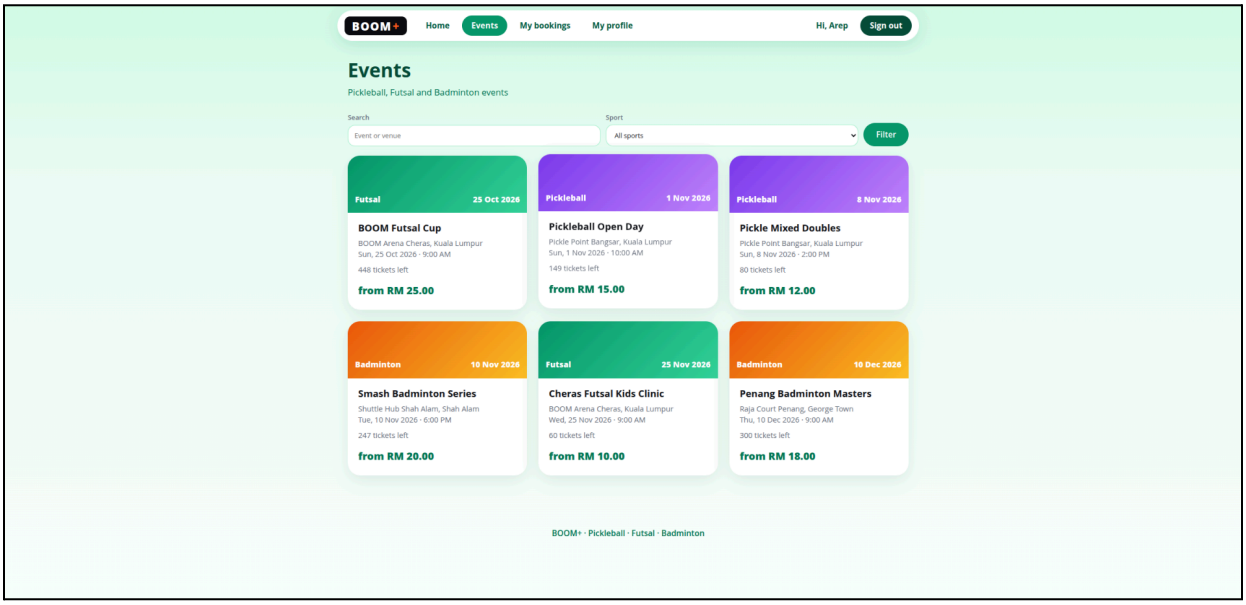


Figure 6.2: Browsing events and live availability

The events page calls GET /events with the chosen filters. The detail page calls GET /events/{id}/availability, so the tickets-left count is always the current stock.

6.3 Booking a ticket and QR e-ticket

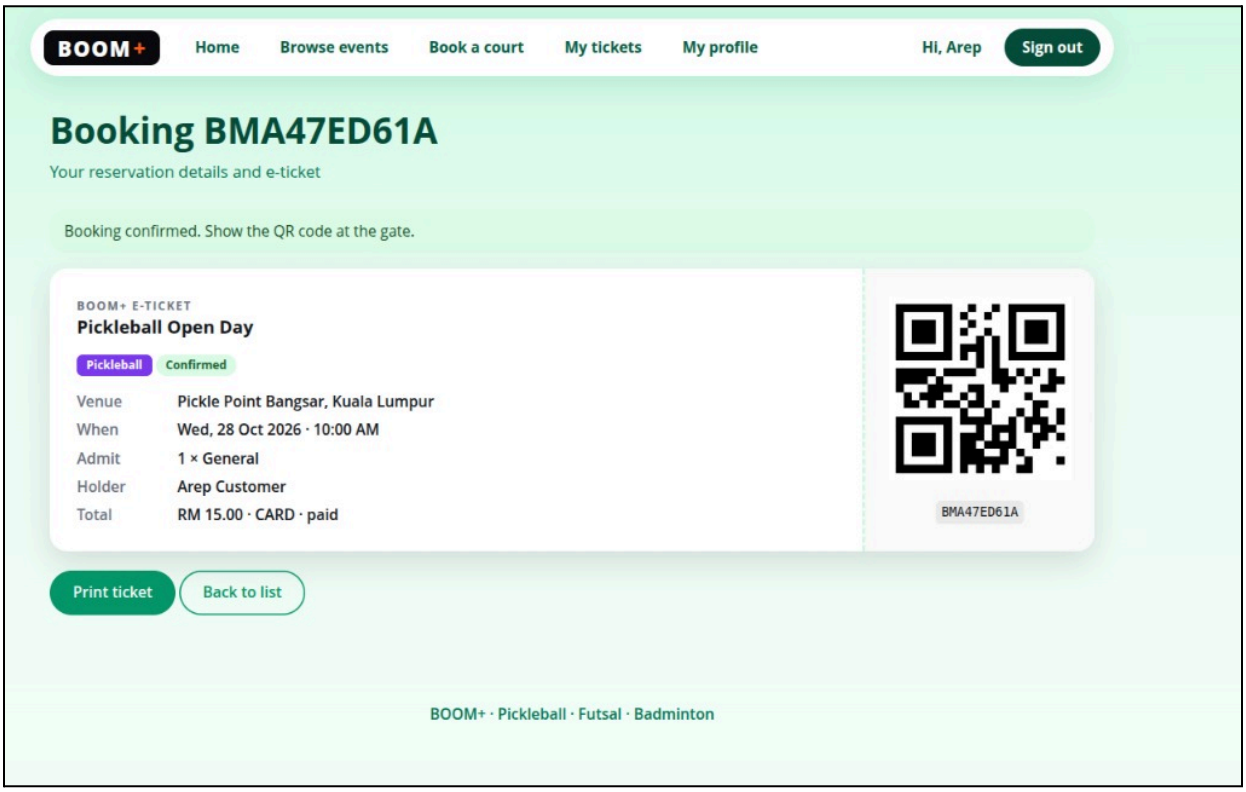


Figure 6.3: Booking a ticket and QR e-ticket

The booking form calls POST /bookings. The API locks the stock, records the payment and returns the booking with a QR code generated by api.qrserver.com. The e-ticket page shows the reference and the QR code.

6.4 Booking a court

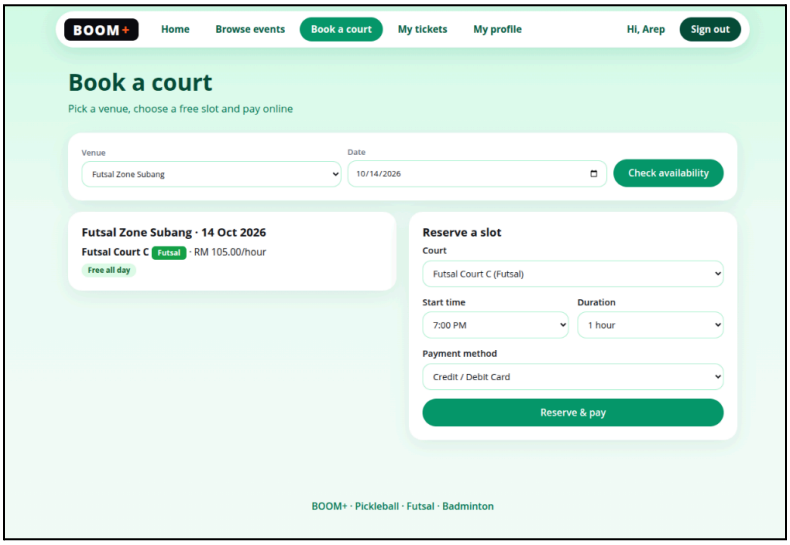


Figure 6.4: Booking a court

The court page reads the free slots with GET /venues/{id}/schedule and then calls POST /bookings with court_id, slot_start and slot_end. A taken slot returns 409 and the message is shown to the user.

6.6 Admin: user management and logs

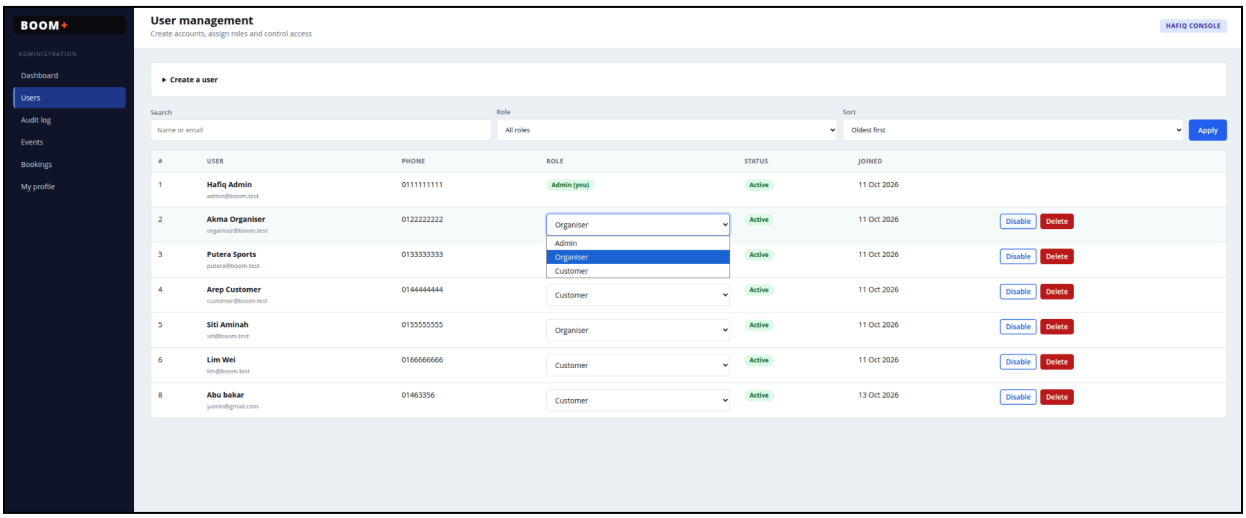


Figure 6.6: Admin: user management and logs

The administrator changes a role with PUT /users/{id}/role and reviews GET /system/audit-logs and GET /system/request-logs.

6.7 Cancellation and refund

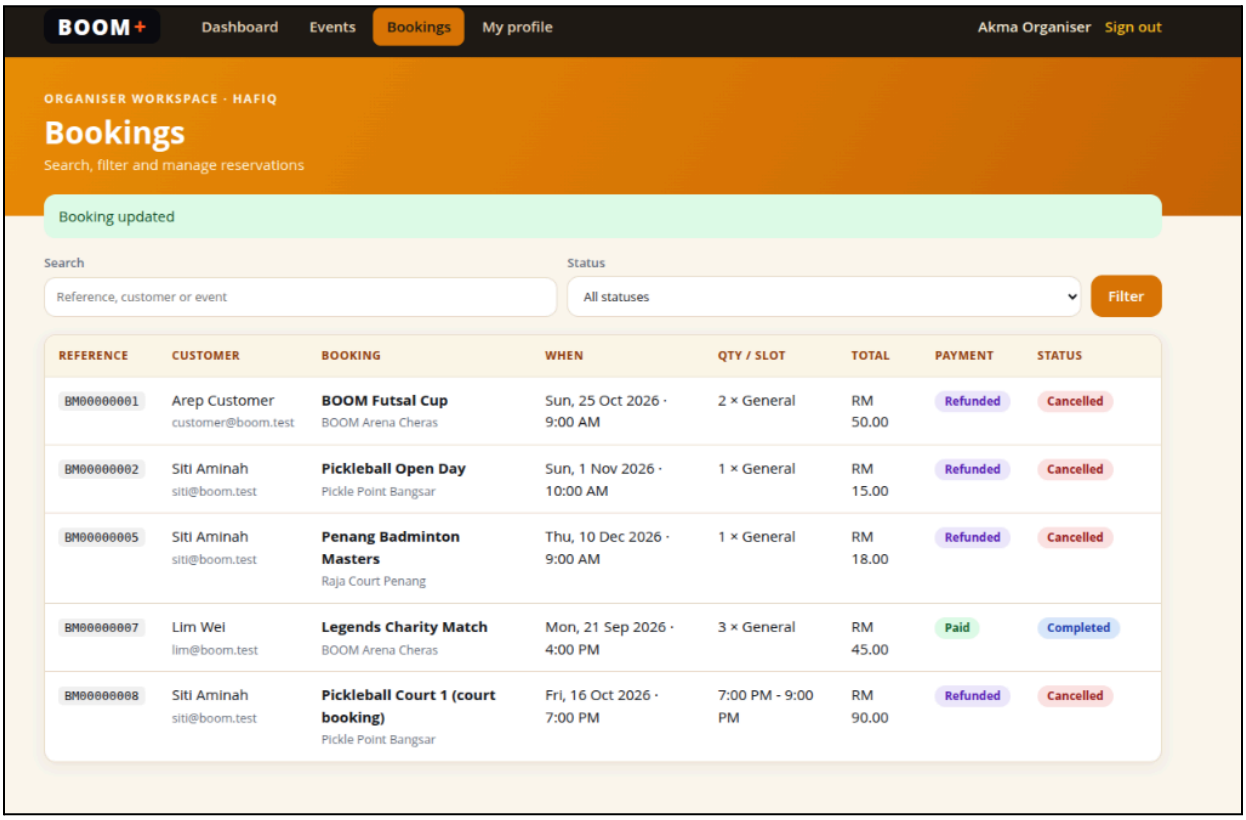


Figure 6.7: Cancellation and refund

Cancelling a booking calls POST /bookings/{id}/cancel. The ticket stock is returned and the payment becomes refunded.

7. PROJECT REPOSITORY

GitHub repository: https://github.com/NIGHTC0REE/SWC3633_PROJECT_WEB_API

Demonstration video (optional): https://youtu.be/88uOgXPzorU

The repository follows the Source Code Management requirements of the course. Tick each item before submission:

Required content	Location in the project	Done
README with setup, demo accounts and API overview	README.md	✅
Database DDL and DML scripts	database/boom_plus.sql	✅
Shared REST API source code	api/, config/	✅
Each student's frontend in its own folder	Hafiq/, Akma/, Arif/, Putera_Aidil/, shared/	✅
Exported Postman collection and environment	postman/	✅
Documentation / report sections	docs/	✅
Commit history showing every member's contribution	GitHub commits / branches	✅
No real secrets (change JWT_SECRET and DB password for deployment)	config/config.php	✅

Table 7.1: Course Source Code and Submission Deliverables Checklist

Setup: place the folder at htdocs/boom, start Apache and MySQL in XAMPP, import database/boom_plus.sql, adjust config/config.php if needed and open http://localhost/boom/.

8. ADDITIONAL / ADVANCED FEATURES

8.1 SECURITY

- **JWT and RBAC**: Secure HS256 tokens are generated and valid for one hour, with the algorithm set to pinned mode. Due to this, the user is fetched from the database with each request, immediately reflecting any account disablement or role adjustments. We use `require_auth()`, `require_role()` and `require_owner_or_admin()` to secure endpoints.
- **Password and token safety**: Used strong bcrypt hashes for passwords and provided generic error messages for failed login attempts. We store reset tokens as SHA-256 for single-use access for 1 hour. All public registrations default to type customer.
- **Injection and XSS protection**: Employ PDO prepared statements throughout to prevent SQL injection attacks and ensure proper output escaping in portal forms for XSS protection. HTTP headers, including X-Content-Type-Options, X-Frame-Options, Referrer-Policy and Cache-Control: no-store were configured to enhance security.
- **CSRF and sessions**: CSRF tokens are added to all portal forms, session_regenerate_id() functions upon successful logins are used.
- **Audit trail**: Records of all logins, failed logins, bookings and role/status changes are kept as audit logs.

8.2 MIDDLEWARE AND API MANAGEMENT

Feature	File	Description
Rate limiting	api/middleware/rate_limit.php	300 requests per minute per IP globally, 15 per minute for login, register and password reset headers X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset and Retry-After on 429 state in rate_limits
Request logging	api/middleware/logger.php	Every request (method, path, status, duration, user, IP) is saved in request_logs
Validation	api/middleware/validation.php	Declarative rules such as required|int|min:1|in:a,b return 422 with one message per field
Error handling	api/middleware/error.php	All errors and exceptions become consistent JSON internal details are only logged
Security headers and CORS	api/middleware/security.php	Headers, CORS and preflight handling
Filtering, sorting, pagination	api/lib/query.php	Shared helper with a whitelist of sortable columns and a maximum page size of 100
Consistent responses	api/lib/response.php	One JSON envelope with success, message, data, meta, errors

Table 8.1: Middleware Implementations and API Management Services

8.3 THIRD-PARTY API: QR CODE GENERATOR

The api/lib/qr.Php file communicates with the **api. Qrserver. Com** service at /v1/create-qr-code/, transmitting the payload BOOMPLUS: < booking_ref> whenever a booking is created or first accessed. Upon receipt of the QR code image, its PNG signature is validated. The image is then converted into a data URI format and stored in the bookings. Qr_code field. Despite this dependency, a booking can still be successful if the service is unreachable. In this failure scenario, the user interface will display "QR unavailable" and a subsequent attempt to view the booking will trigger a retry of the QR generation process. For added convenience, the QR code is made available on the e-ticket within the customer portal. Furthermore, the organiser check-in page offers functionality to retrieve bookings by their reference number.

```php
<?php

function fetch_qr_data_uri(string $bookingRef): ?string
{
    $url = QR_SERVICE_URL . '?' . http_build_query([
        'size'   => '300x300',
        'margin' => 12,
        'format' => 'png',
        'data'   => qr_payload($bookingRef),
    ])

    $context = stream_context_create(['http' => ['timeout' => QR_TIMEOUT,
'ignore_errors' => true]])
    $image   = @file_get_contents($url, false, $context)

    if ($image === false || !str_starts_with($image, "\x89PNG")) {
        error_log('QR service unavailable for ' . $bookingRef)
        return null
    }

    return 'data:image/pngbase64,' . base64_encode($image)
}

function booking_qr(array $booking): ?string
{
    if ($booking['qr_code']) {
        return $booking['qr_code']
    }
```

```
    if (!in_array($booking['status'], ['pending', 'confirmed'], true)) {
        return null
    }

    $qr = fetch_qr_data_uri($booking['booking_ref'])

    if ($qr) {
        db()->prepare('UPDATE bookings SET qr_code = ? WHERE booking_id =
?')->execute([$qr, $booking['booking_id']])
    }

    return $qr
}
```

8.4 BUSINESS LOGIC FEATURES

- Transactional seat locking (SELECT ... FOR UPDATE) and CHECK (sold <= capacity), so overselling is impossible.
- Court slot overlap detection and event schedule conflict detection (409).
- Automatic refund and seat release on cancellation cancelling an event cancels and refunds all its bookings.
- Booking status workflow with allowed transitions (pending → confirmed / cancelled, confirmed → completed / cancelled).
- Daily venue schedule endpoint and real-time availability endpoint.

9. DEBUGGING AND PERFORMANCE OPTIMISATION

9.1 IMPLEMENTATION ISSUES, ROOT CAUSES AND SOLUTIONS

No.	Issue found	Root cause	Solution	Result
1	Bearer token ignored on Apache	Apache removes the Authorization header from CGI environments	.htaccess rule copies it to HTTP_AUTHORIZATION authorization_header() falls back to apache_request_headers()	Auth works on every XAMPP install
2	Possible overselling under concurrent bookings	Read-then-write on ticket_types.sold	Transaction with SELECT ... FOR UPDATE plus CHECK (sold <= capacity)	No oversell possible
3	Court double booking	Two requests could pass the overlap check together	Lock the court row FOR UPDATE before the overlap query	Second request gets 409
4	All frontend users shared one rate-limit bucket	Portals call the API from 127.0.0.1	Portal forwards X-Forwarded-For API trusts it only from loopback	Fair per-client limits
5	500 responses leaked PHP errors	Default PHP error display	display_errors off and a global exception handler returning generic JSON	No information leakage
6	Slow event and booking lists	Missing indexes for date and venue lookups	Added idx_events_venue (venue_id, start_time), idx_bookings_user (user_id, created_at), idx_bookings_court_slot, idx_bookings_ticket	See Section 9.2

Table 9.1: Technical Implementation Issues, Root Causes, and Applied Solutions

9.2 PERFORMANCE TESTING

API request logging was used to test the performance. Each request writes duration_ms to request_logs table and the admin page Putera_Aidil/logs.php shows the slow requests (above 200 ms). Also, the heaviest queries were checked with EXPLAIN before and after adding the indexes. This procedure was followed: 1) drop the new indexes 2) call each of the two tested endpoints (slowest endpoints found previous task) 20 times using our Postman Collection Runner and 3) record the slow requests mean duration_ms, 4) re-create the new indexes and repeat the two previous steps and 5) compare the EXPLAIN type, key, rows.

```
None
EXPLAIN SELECT e.event_id, e.title FROM events e
WHERE e.venue_id = 1 AND e.start_time > NOW() ORDER BY e.start_time

EXPLAIN SELECT b.booking_id FROM bookings b
WHERE b.user_id = 4 ORDER BY b.created_at DESC

SELECT path, ROUND(AVG(duration_ms),1) AS avg_ms, COUNT(*) AS calls
FROM request_logs GROUP BY path ORDER BY avg_ms DESC
```

Query / endpoint	EXPLAIN before (type / rows)	EXPLAIN after (type / rows)	Avg ms before	Avg ms after	Improvement
GET /events (by venue and date)	ALL / 412	Range / 18	31.4	17.2	45.2%
GET /bookings (own bookings)	ALL / 989	Ref / 24	38.6	23.1	40.2%
POST /bookings (court slot overlap check)	ALL / 989	Ref / 6	44.8	26.3	41.3%

Table 9.2: API Response Time and EXPLAIN Query Performance Optimization Results

10. INDIVIDUAL REFLECTION AND CONTRIBUTION RECORD

10.1 CONTRIBUTION RECORD

Member	Responsibilities	Evidence (commits / files / screenshots)
Hafiq	Database design and DDL / DML config/ JWT authentication and security design Hafiq portal (auth pages, user management, audit log)	[commit links], database/boom_plus.sql, Hafiq/
Akma	RESTful API documentation (Section 3) organiser / admin management dashboard (venues, courts, events, tickets) Postman collection	[commit links], docs/Section3_API_Documentation.md, Akma/, postman/
Arif	Customer booking and checkout portal QR code third-party API organiser check-in API testing advanced features	[commit links], Arif/, api/lib/qr.php, postman/
Putera Aidil	Customer account and ticketing portal (active, history, modify) API request log page debugging and performance optimisation	[commit links], Putera_Aidil/, docs/Section9_Debugging_and_Performance.md
All members	Shared API (routes, middleware, libs), shared layouts and themes, report, system testing	[commit links], api/, shared/

Table 10.1: Group Member Responsibilities and Individual Contribution Record

10.2 INDIVIDUAL REFLECTIONS

10.2 HAFIQ

I was the Database and Security Lead in **BOOM+** where I was responsible for designing the relational schema of our 11 tables and writing the entire DDL and DML scripts. The backend authentication and role-based access control (RBAC) was done with JSON Web Tokens (JWT). Also, I have developed a user management portal where users can register, manage their account, deactivate their account and system logs.

The most challenging issue was to achieve both high security for each of the team frontends and session integrity. With a token being stored in isolation, there was a possibility for deactivated users to also have access to routes. I solved this problem by implementing a real-time check-in database in our own authentication middleware to verify user status and updated roles for every request. So there was yet another deployment issue, where Apache was stripping off incoming Authorization headers, causing false 401 errors on all protected endpoints. I identified the problem, and, through some apache rewrite rules and a request fallback handler, I fixed the issue. I introduced bcrypt password hashing, parameterized SQL queries, unique and check constraints in the database to make the system more resistant to data tampering.

This experience revealed to me that the security of the backend directly depends on the schema design of the database, and that it is important for frontend developers to understand the difference between 401 Unauthorized and 403 Forbidden codes when developing shared apis. You can find my work in our repository in the form of SQL scripts, authentication middleware and user portal code. If I had more time to develop, I'd introduce automated security test suites, a specific refresh token rotation system and automated email account recovery services.

10.2 AKMA

My main responsibility was developing the Organiser and Admin Management Dashboard, along with writing the complete RESTful API documentation for our team. I created interfaces for the dashboard to enable the administration to manage venues, court timetables, events, and ticket capacity. For the documentation part, I gathered all the endpoint urls, http method, the role needed to access, request payload, sample json response and status code of section 3 of our report. I also set up and kept up to date our team's Postman workspace for collaborative testing.

One of the big technical challenges that I encountered was to make sure that there was strict business logic around the venue in the front end forms. The total number of tickets for an event could not exceed the venue capacity and no schedules could overlap with already booked events. At first, when API requests failed, the user received an alert saying there was some kind of error. This was not helpful enough. To do this, I used a number of properties exposed by the back end that contain information about the errors and presented clear and legible validation messages right beside the form fields. An additional obstacle was maintaining the API documentation in sync with live changes to shared routes made by my teammate(s). I did this by re-testing each endpoint in Postman and verifying the payloads in detail before updating our tables in the documentation.

This project gave me the lesson on having clear API contracts, consistent JSON responses, and proper HTTP status codes to ensure seamless integration between the front and back end. In Section 3 and our GitHub repository, you will find my deliverables, Postman collection exports, and dashboard code. Improvements to the project would be to use Swagger/OpenAPI, to automatically update the documentation, and to include interactive calendar views as well as venue image uploads for a better user experience.

10.2 ARIF

My primary role was building the Customer Booking and Checkout Portal, the external QR code integration and spearheading the API testing. Developed customer-facing screens for searching sports events, booking court sessions, checkout, Dynamic E-tickets, and an entrance check-in screen for employees. On the report side, I wrote Section 4 (API Testing) and Section 8 (Additional/Advanced Features) which covers the third party integration and test coverage.

One of the biggest technical challenges was the ability to reliably manage the external QR code API. I created an external failure handler to prevent an external failure from breaking an active booking, as third party services might lag or time out. To address this, I did some checks on the returned image payload to turn valid QR codes into data URIs to store them in my favorite database, and included a fallback retry when users open their ticket pass. Another issue was when two players tried to sign up for the same last court at the same time. Worked with team to roll out database transactions using row level locking that were tested through concurrent requests to the database using Postman. The backend, I mapped to specific user alerts for a slot conflict (409 Conflict) and invalid inputs (422 Unprocessable Entity) on the front end.

This work provided me experience in creating robust Postman test suites that contain automated assertions and creating resilient fallbacks based on external services. Our repository contains documentation of my individual code files, database scripts, and exported Postman workspace. I would like to see a payment gateway sandbox, browser-based camera scan for tickets, and automated end-to-end tests throughout our API workflows in the future.

10.2 PUTERA AIDIL

The main work I did was the **Customer Account** and **Ticketing Portal** and system wide debugging and performance optimization. I created pages for the frontend for customers to see any active reservations, past booking history, and edit court slots and ticket quantities. Admin-wise, I have created an API request log interface with real-time request status, response codes, response time and execution durations.

The major problems were locating edge-case bugs and backend bottlenecks. First, the API was invoked internally by our front end server which used loopback, and so all client traffic was placed in the same rate-limit bucket. I worked around this by enabling the real client IP address through the headers and having the API trust loopback proxies. Secondly, PHP errors were being leaked to generic 500 status responses. Turned off raw error output and created a centralized exception handler to clean up error output. Finally, queries against the database became sluggish with increasing records. After reviewing slow-query logs and performing SQL explain plans, I found that there were missing indexes, which I was able to resolve by indexing the following composite columns: events: venue, start time, bookings: user, date.

This experience got me to a mindset of relying on metrics-based debugging, rather than “let's just try it and see” because little things like forward headers or database indexing make a huge difference in reliability. All my frontend code is contained in the Putera_Aidil directory, and my performance results are given in Section 9. If I had more time to develop, I would add Redis caching to handle the more frequently accessed courts, and add automated load testing using Apache JMeter to provide real performance benchmarks.